QF5204 Numerical Methods in Quantitative Finance

AY2025/26 Sem2 By Zhao Peiduo

Lecture 1

Numerical Methods in the Financial Industry Numerical methods play a critical role in modeling, analyzing, and solving complex problems in finance, especially when analytical solutions are infeasible. They are widely applied in risk management, derivative pricing, portfolio optimization, and market simulation.

Key Applications Option pricing and hedging, risk management, portfolio optimization, fixed-income analytics, algorithmic trading.

What Is a Numerical Method A numerical method focuses on obtaining approximate solutions to mathematical problems when exact, closed-form solutions are unavailable or impractical.

Main Components

- Theoretical development of numerical schemes
- Practical implementation as efficient and reliable algorithms

How Numerical Methods Work Simplify complex problems to tractable approximations and solve them numerically. These methods rely on tools from linear algebra, real analysis, and functional analysis. A central focus is understanding and controlling error, including error magnitude, analytical bounds, efficiency, and computational cost.

Core Considerations

- **Accuracy:** ensuring results are precise and reliable
- **Computational cost:** balancing efficiency with time and resource constraints

Python Toolbox

- Packages: numpy, scipy, matplotlib, pandas, statsmodels, quandl, matplotlib
- Techniques: Reshaping, reference vs copy, j for complex number, indexing and slicing, upcasting, broadcasting, vectorization, lambda expressions.
- Practices: Modularization and organization, error handling, computational cost estimation

Errors in Numerical Methods Numerical methods produce approximate solutions when analytical solutions are unavailable, making error analysis central to reliability.

Types of Errors

- **Modeling error:** mismatch between physical reality and simplified mathematical model
- **Truncation error:** approximation of infinite processes by finite ones
- **Round-off error:** finite precision limitations in computer arithmetic

Approximation Pipeline

- Problem $\to$ mathematical model (modeling error)
- Model $\to$ numerical method (truncation error)
- Method $\to$ computation (rounding error)

Numerical Error Measures

- **True error:** $E_t = \text{true value} - \text{approximation}$
- **Relative error:** $\dfrac{E_t}{\text{true value}}$
- **Relative error percentage:** $\dfrac{E_t}{\text{true value}} \times 100\%$

Binary Number Representation Computers use binary representations composed of bits $\{0, 1\}$

- Positive integer:

$$d_n d_{n-1} \cdots d_0 = \sum_{i=0}^{n} d_i 2^i$$

- Fractional number:

$$d_{-1} d_{-2} \cdots d_{-m} = \sum_{i=1}^{m} d_{-i} 2^{-i}$$

Integer Ranges (32-bit) Unsigned: $[0, 2^{32} - 1]$; Signed: $[-2^{31}, 2^{31} - 1]$

Fixed-Point Representation

- Binary point is fixed at a predefined position
- Defined by width w and binary point position b
- Denoted as $\text{fixed}\langle w, b\rangle$; Integer type: $\text{fixed}\langle 32, 0\rangle$
- Limited range $\Rightarrow$ overflow error when exceeded

Fixed-Point Value Formula

For $\text{fixed}\langle w, b\rangle$ with bits $d_{w-1} \cdots d_0$:

$$x = \begin{cases} \displaystyle\sum_{k=0}^{w-2} d_k 2^{k-b}, & \text{if } d_{w-1} = 0 \quad \text{(positive real number)}, \\ & \text{e.g., } 0111_2 \text{ in } \text{fixed}\langle 4, 1\rangle : \ x = 2 + 1 + 0.5 = 3.5, \\ \displaystyle -2^{w-1-b} + \sum_{k=0}^{w-2} d_k 2^{k-b}, & \text{if } d_{w-1} = 1 \quad \text{(negative real number)}, \\ & \text{e.g., } 1011_2 \text{ in } \text{fixed}\langle 4, 1\rangle : \ x = -4 + 1 + 0.5 = -2.5 \end{cases}$$

Floating-Point Representation (IEEE 754) Binary scientific notation

- Structure: sign bit, exponent bits, significand bits
- Value:

$$(-1)^s (1 + \text{significand}) 2^{\text{exponent} - \text{bias}} \text{(e.g.} 1.01 \times 2^{-1}, \ 1{,}01 \text{ Significand; -1 Exponent)}$$

Floating-Point Ranges 32-bit float: $\approx 10^{-38}$ to 10^{38}; 64-bit float: $\approx 10^{-308}$ to 10^{308}

Numerical Errors Definitions

- True error: $E_t = \text{True value} - \text{Approximation}$
- Relative error: $E_r = \dfrac{|E_t|}{|\text{True value}|}$
- Relative error percentage: $\%\ \text{error} = E_r \times 100\%$

Number Representation and Range

- **Binary representation:** integers use bits $d_n \cdots d_0$ with value $\sum_{k=0}^{n} d_k 2^k$.
- **32-bit integer ranges:** unsigned $[0, 2^{32} - 1]$, signed $[-2^{31}, 2^{31} - 1]$.
- **Fixed-point:** type $\text{fixed}\langle w, b\rangle$ has width w and binary point at position b; limited range $\Rightarrow$ overflow possible.
- **Floating-point:** scientific notation in base 2 (IEEE 754); wide dynamic range. Typical ranges: 32-bit float $\approx 10^{-38}$ to 10^{38}, 64-bit float $\approx 10^{-308}$ to 10^{308}.

Rounding Error Effects

- Errors accumulate gradually across computations
- Subtracting nearly equal numbers causes catastrophic cancellation
- Adding numbers of very different magnitudes causes loss of precision

Machine Epsilon

- Machine epsilon ε is the smallest number such that $1 + \varepsilon \neq 1$ in machine arithmetic.
- It quantifies round-off effects and depends on hardware/precision.

Truncation Error (Taylor-based) Truncation error comes from dropping higher-order terms.

$$f(x + h) = f(x) + hf'(x) + \frac{h^2}{2} f''(x) + \cdots$$

$$f(x) = f(a) + f'(a)(x - a) + \cdots + \frac{f^{(n)}(a)}{n!}(x - a)^n + R_n(x), \qquad R_n(x) = \frac{f^{(n+1)}(c)}{(n + 1)!}(x - a)^{n+1}.$$

Local vs. Global Truncation Error

- **Local truncation error:** error introduced in one step; for first-order Taylor,

$$f(x + h) = f(x) + hf'(x) + O(h^2), \quad \Rightarrow \text{local error } = O(h^2).$$

- A method has order p if local truncation error is $O(h^{p+1})$.
- **Global truncation error:** accumulation over K steps with $\Delta h = h/K$:

$$e_K \approx K \cdot O(\Delta h^2) = \frac{h}{\Delta h} O(\Delta h^2) = O(h\Delta h).$$

- Convergent methods satisfy global error $\to 0$ as $\Delta h \to 0$.

Convergence and Rate

- $\{x_k\}$ converges to x^* if $\lim_{k\to\infty} x_k = x^*$.
- With error $e_k = |x_k - x^*|$, order p if

$$\lim_{k\to\infty} \frac{|e_{k+1}|}{|e_k|^p} = c_0.$$

Common Convergence Behaviors

- **Linear ($p = 1$):** $\|x_{k+1} - x^*\| \le c_0 \|x_k - x^*\|, \ 0 < c_0 < 1$.
- **Sub-linear:** Logarithmic Convergence: $\|x_k - x^*\| = O(1/\log k)$ or Polynomial Convergence: $O(1/k^p)$.
- **Super-linear:** $\dfrac{\|x_{k+1} - x^*\|}{\|x_k - x^*\|} \to 0$.
- **Quadratic ($p = 2$):** $\dfrac{\|x_{k+1} - x^*\|}{\|x_k - x^*\|^2} = c_0$

Tolerance and Stopping Criteria

- True error requires knowing the true value:

$$E_t = \text{True value} - \text{Approximation}.$$

- In practice, the true value is unknown, so we use tolerance-based tests to decide when an iterate is "close enough".

Typical tests (tolerance ε_0) Choice of test depends on the application (scaling, units, noise sensitivity).

- **Absolute change:** $|x_{k+1} - x_k| \le \varepsilon_0$.
- **Relative change:** $\dfrac{|x_{k+1} - x_k|}{|x_k|} \le \varepsilon_0$.

Lecture 2

The Time Value of Money

- An amount x invested at interest rate r grows to $x(1 + r) = x + xr$, where x is the principal and xr is the interest.
- With continuous compounding for n years, the value becomes xe^{rn}.
- A *fixed-income security* provides a predetermined stream of cash flows.
- *Annuities*: pay a fixed amount A at the end of every period indefinitely.
- *Bonds*: pay a face (par) value at a specified maturity time.

Introduction to Forward Contracts

- A *derivative* is a security whose payoff depends on another variable.
- A *forward contract* fixes today the price of an asset delivered at future time T.
- Two times are fixed: 0 (today) and T (delivery). $F(0, T)$ denotes the forward price agreed upon today.
- *Long position*: agree to buy at T for $F(0, T)$.
- *Short position*: agree to sell at T for $F(0, T)$.

No-Arbitrage Condition in Forward Pricing

- Arbitrage arises if the forward price deviates from its fair theoretical value.
- *Cash-and-carry arbitrage* (forward overpriced): borrow at rate r, buy spot asset at S, short the forward, profit $F - Se^{rT}$.
- *Reverse cash-and-carry* (forward underpriced): short the asset, invest proceeds, go long the forward, profit $Se^{rT} - F$.
- No-arbitrage implies the cost-of-carry formula $F = Se^{rT}$.

Dynamic (Tree-Based) Models

- Discrete-time price dynamics are commonly modeled using trees.
- Asset values evolve multiplicatively: $S \to uS$ (up) or dS (down).
- Constants u and d are fixed multipliers.
- Let p be the probability of an up move and $1 - p$ of a down move.

Binomial Tree Parameters

- Define yearly expected growth rate and volatility:

$$\mu = \mathbb{E}[\ln(S(1)/S(0))], \quad \sigma = \sqrt{\mathrm{Var}(\ln(S(1)/S(0)))}.$$

- Choose parameters: Let Δt be the time step (typically a day or a week). We model the log-price increment over one time step Δt by a binomial move:

$$\log \frac{S_{t+\Delta t}}{S_t} = \begin{cases} \log u, & \text{with probability } p, \\ \log d, & \text{with probability } 1-p. \end{cases}$$

Choose symmetric up/down factors

$$u = e^{\sigma\sqrt{\Delta t}}, \quad d = e^{-\sigma\sqrt{\Delta t}} \quad \Rightarrow \quad \log u = \sigma\sqrt{\Delta t}, \quad \log d = -\sigma\sqrt{\Delta t}.$$

The expected log-return is then

$$\mathbb{E}\left[\log \frac{S_{t+\Delta t}}{S_t}\right] = p \log u + (1-p)\log d = (2p-1)\sigma\sqrt{\Delta t}.$$

To match the drift μ of the continuous-time log-price,

$$\mathbb{E}\left[\log \frac{S_{t+\Delta t}}{S_t}\right] \approx \mu\,\Delta t \Rightarrow (2p-1)\sigma\sqrt{\Delta t} = \mu\,\Delta t.$$

Solving for p yields

$$p = \frac{1}{2} + \frac{1}{2}\frac{\mu}{\sigma}\sqrt{\Delta t}.$$

General Discrete-Time Models

- Consider the multiplicative model

$$S(k+1) = u(k)S(k),$$

where $\{u(k)\}$ are independent random variables.
- Taking logarithms,

$$\ln S(k+1) = \ln S(k) + \ln u(k).$$

- If $X \sim \mathcal{N}(\mu, \sigma^2)$, then $Y = e^X$ is lognormally distributed:

$$Y \sim LN(\mu, \sigma^2), \quad \text{and} \quad \mathbb{E}[Y] = e^{\mu+\sigma^2/2}.$$

- In particular, in a lognormal multiplicative model with $u(k) \sim LN(\nu, \sigma^2)$, the log-returns satisfy

$$\mathbb{E}\left[\ln \frac{S(k+1)}{S(k)}\right] = \mathbb{E}[\ln u(k)] = \nu \quad \mathrm{Var}\left[\ln \frac{S(k+1)}{S(k)}\right] = \mathrm{Var}[\ln u(k)] = \sigma^2.$$

Random Walks and Wiener Process

- Discrete stochastic process:

$$z(t_{k+1}) = z(t_k) + \epsilon(t_k)\sqrt{\Delta t}, \quad \epsilon(t_k) \sim \mathcal{N}(0,1).$$

- As $\Delta t \to 0$, $z(t)$ converges to the Wiener process:

$$dz(t) = \epsilon(t)\sqrt{dt}.$$

- Properties of Brownian motion: $z(0) = 0$, continuous paths,

$$z(t) - z(s) \sim \mathcal{N}(0, t-s), \quad t > s.$$

Ito Process and Geometric Brownian Motion

- Wiener process with drift:

$$dx(t) = a\,dt + b\,dz(t), \quad a, b \in \mathbb{R}.$$

- Integral form:

$$x(t) = x(0) + at + bz(t).$$

- General Ito process:

$$dx(t) = a(t)\,dt + b(t)\,dz(t).$$

- Geometric Brownian motion (GBM) for stock prices:

$$dS(t) = \mu S(t)\,dt + \sigma S(t)\,dz(t).$$

Ito's Lemma

- Let $x(t)$ follow an Ito process and $G(x,t)$ be a smooth function.
- Taylor expansion (ignoring higher-order terms):

$$\Delta G = \frac{\partial G}{\partial x}\Delta x + \frac{\partial G}{\partial t}\Delta t + \frac{1}{2}\frac{\partial^2 G}{\partial x^2}(\Delta x)^2.$$

- With $\Delta x = a\Delta t + b\epsilon\sqrt{\Delta t}$, $\epsilon \sim \mathcal{N}(0,1)$ and $\mathbb{E}[\epsilon^2] = 1$,
- Taking limits yields Ito's lemma:

$$\Delta G = \frac{\partial G}{\partial x}\Delta x + \frac{\partial G}{\partial t}\Delta t + \frac{1}{2}\frac{\partial^2 G}{\partial x^2}b^2\epsilon^2\Delta t \quad \Rightarrow \quad \Delta G = \frac{\partial G}{\partial x}\Delta x + \frac{\partial G}{\partial t}\Delta t + \frac{1}{2}\frac{\partial^2 G}{\partial x^2}b^2\Delta t$$

Ito's Lemma for Stock Prices

- Taking limits, for a function $G(x,t)$:

$$dG = \frac{\partial G}{\partial x}\,dx + \frac{\partial G}{\partial t}\,dt + \frac{1}{2}\frac{\partial^2 G}{\partial x^2}b^2\,dt.$$

- Substituting $dx = a\,dt + b\,dz$, we obtain Ito's lemma:

$$dG = \left(\frac{\partial G}{\partial x}a + \frac{\partial G}{\partial t} + \frac{1}{2}\frac{\partial^2 G}{\partial x^2}b^2\right)dt + \frac{\partial G}{\partial x}b\,dz.$$

- Application to the stock price process:

$$dS = \mu S\,dt + \sigma S\,dz.$$

- For a function $G(S,t)$:

$$dG = \left(\frac{\partial G}{\partial S}\mu S + \frac{\partial G}{\partial t} + \frac{1}{2}\frac{\partial^2 G}{\partial S^2}\sigma^2 S^2\right)dt + \frac{\partial G}{\partial S}\sigma S\,dz.$$

Options

- An option is a financial contract whose value depends on an underlying asset.
- The holder has the right, but not the obligation, to trade at a fixed price.
- Strike price: K, maturity time: T.
- Call option: right to buy at time T for price K.
- Put option: right to sell at time T for price K.
- Payoffs at maturity:

$$\text{Call: } \max(S_T - K, 0), \qquad \text{Put: } \max(K - S_T, 0).$$

- Derivative payoff is a function of the underlying: $X = F(S_T)$.

Option Strategies

- Option strategies combine basic options to create desired payoffs.
- Strangle: buy a put at K_1 and a call at $K_2 > K_1$.
- Straddle: special case of strangle with $K_1 = K_2$.
- Bull spread: buy call at K_1, sell call at $K_2 > K_1$.
- Bear spread: sell put at K_1, buy put at $K_2 > K_1$.

Option Moneyness

- Moneyness describes the relation between stock price S and strike K.
- In-the-Money (ITM): option has positive intrinsic value if exercised now.
- At-the-Money (ATM): $S \approx K$.
- Out-of-the-Money (OTM): option has zero intrinsic value if exercsised.

Financial Derivatives

- A financial derivative is a contract whose value depends on an underlying asset (stocks, bonds, commodities).
- Types of derivatives:
  - Vanilla options: standardized, simple payoff structures.
  - Exotic options: complex features and conditions.
- Linear products include forwards, futures, and swaps.
- **Law of One Price**: identical assets must have the same price in frictionless markets.

European Options and Black–Scholes Model

- European options may only be exercised at expiry.
- Priced analytically using the Black–Scholes model.
- Assumptions:
  - No arbitrage opportunities.
  - Constant risk-free interest rate.
  - Log-normal distribution of underlying prices.
  - No dividends.
  - Frictionless markets.

Pricing Exotic Options

- Exotic options have complex or path-dependent payoffs.
- Examples:
  - American options: exercisable anytime before expiry.
  - Barrier options: activated when price hits a barrier.
  - Asian options: payoff depends on average price.
- Analytical pricing may fail due to complexity.
- Numerical approaches:
  - Binomial tree models.
  - Monte Carlo simulation.
  - Finite difference methods.

Binomial Tree Model

- Discrete-time model for option pricing.
- Asset price moves up or down at each step.
- Proposed by Cox, Ross, and Rubinstein (1979).
- Advantages: simple, applicable to European and American options.

One-Step Binomial Model

- Time interval $[0, T]$ with two possible states at T.
- Initial stock price: S_0.
- Up state: $S_u = uS_0$, with probability p.
- Down state: $S_d = dS_0$, with probability $1 - p$.
- Risk-free interest rate: r.

Pricing by Replication

- Portfolio consists of δ shares of stock and cash B.
- Initial value:
$$V_0 = \delta S_0 + B.$$
- Replication conditions:
$$\delta S_u + e^{rT} B = V_u, \quad \delta S_d + e^{rT} B = V_d.$$

Option Payoffs

- Call option:
$$V_u = \max(S_u - K, 0), \quad V_d = \max(S_d - K, 0).$$
- Put option:
$$V_u = \max(K - S_u, 0), \quad V_d = \max(K - S_d, 0).$$

Solving the Linear System

- Hedge ratio:
$$\delta = \frac{V_u - V_d}{S_u - S_d}.$$
- Cash position:
$$B = e^{-rT}(V_u - \delta S_u).$$
- Option price:
$$V_0 = e^{-rT}(q_u V_u + q_d V_d),$$
where
$$q_u = \frac{S_0 e^{rT} - S_d}{S_u - S_d}, \quad q_d = \frac{S_u - S_0 e^{rT}}{S_u - S_d}.$$

Implementation

- Inputs:
  - Stock price S_0, interest rate r.
  - Up/down factors u, d.
  - Option type, strike K, maturity T.
- Output: option price.
- One-step binomial pricing demonstrated for call and put.
- In practice, u and d are chosen using volatility σ.

Is the Binomial Tree Model Still Useful?

- The binomial tree is a simplified, step-by-step framework for understanding option price evolution.
- It provides a discretized approximation of the continuous Black–Scholes model.
- It avoids explicitly fixing S_u and S_d by linking them to volatility σ.
- Stock prices are viewed as following a continuous stochastic process (GBM).

Intuition Behind Black–Scholes

- Stock prices evolve continuously rather than via discrete up/down jumps.
- Two driving components:
  - Drift: average rate of return.
  - Randomness: market uncertainty (volatility).

Black–Scholes Model

- Under the risk-neutral measure, the stock price satisfies
$$dS_t = S_t(r\,dt + \sigma\,dW_t), \qquad \frac{dS_t}{S_t} = r\,dt + \sigma\,dW_t.$$
- Parameters:
  - r: risk-free interest rate,
  - σ: volatility,
  - W_t: standard Brownian motion.
- Assumptions: continuous trading, constant r and σ.

Solution of the Stock Price SDE

- Applying Itô's lemma to $f(S_t) = \ln S_t$:
$$d\ln S_t = \left(r - \tfrac{1}{2}\sigma^2\right)dt + \sigma\,dW_t.$$
- Hence,
$$S_t = S_0 \exp\left(\left(r - \tfrac{1}{2}\sigma^2\right)t + \sigma W_t\right), \quad W_t \sim \mathcal{N}(0, t).$$

Black–Scholes Option Pricing

- Call option price at time 0:
$$C(S_0, K, T) = e^{-rT}\mathbb{E}_Q[(S_T - K)_+].$$
- Defining
$$d_\pm = \frac{\ln(S_0/K) + (r \pm \frac{1}{2}\sigma^2)T}{\sigma\sqrt{T}},$$
the closed-form solution is
$$C(S_0, K, T) = S_0 N(d_+) - Ke^{-rT}N(d_-).$$
- Corresponding put price:
$$P(S_0, K, T) = Ke^{-rT}N(-d_-) - S_0 N(-d_+).$$

Binomial vs. Black–Scholes at Maturity

- Binomial tree: two-point discrete distribution for S_T vs Black–Scholes: continuous lognormal distribution.
- Both models share the same mean:
$$\mathbb{E}[S_T] = S_0 e^{rT}.$$
- Variances differ, but converge as the tree is refined.

Matching the Variance of Two

- One-step binomial model:
$$S_u = uS_0, \quad S_d = dS_0, \quad u > 1,\ 0 < d < 1.$$
- Variance of binomial tree:
$$\mathrm{Var}(S_T) = S_0^2\left(pu^2 + (1-p)d^2\right) - S_0^2 e^{2rT}.$$
- Variance under Black–Scholes:
$$\mathrm{Var}(S_T) = S_0^2 e^{2rT}\left(e^{\sigma^2 T} - 1\right).$$
- Matching variances requires
$$e^{2rT + \sigma^2 T} = pu^2 + (1-p)d^2.$$
- CRR assumption:
$$d = \frac{1}{u}.$$

Risk-Neutral Probability

- No-arbitrage condition:
$$pu + (1-p)d = e^{rT}.$$
- Risk-neutral probability:
$$p = \frac{e^{rT} - 1/u}{u - 1/u}.$$

Solving for u

- Substitution gives:
$$e^{2rT + \sigma^2 T} = e^{rT}\left(u + \frac{1}{u}\right) - 1.$$
- Quadratic equation:
$$u^2 - \left(e^{rT + \sigma^2 T} + e^{-rT}\right)u + 1 = 0.$$
- Roots:
$$u = \frac{b + \sqrt{b^2 - 4}}{2}, \quad d = \frac{b - \sqrt{b^2 - 4}}{2},$$
where $b = e^{rT + \sigma^2 T} + e^{-rT}$.

Bringing It Closer

- One-step variance matching does not fully replicate Black–Scholes.
- Binomial model approximates a continuous stochastic process.
- Accuracy improves by increasing time steps.

Multi-Step Binomial Tree

- Divide $[0, T]$ into N steps with $\Delta t = T/N$.
- State probabilities:
$$p(S_{N,i}) = \binom{N}{i} p^i (1-p)^{N-i}.$$
- Recombining tree ensures polynomial growth of nodes.

Moments of S_T

- First moment:
$$\mathbb{E}[S_T \mid S_0] = S_0(pu + (1-p)d)^N.$$
- Second moment:
$$\mathbb{E}[S_T^2 \mid S_0] = S_0^2(pu^2 + (1-p)d^2)^N.$$

N-Step Matching with Black–Scholes

- Risk-neutral probability:
$$p = \frac{e^{r\Delta t} - d}{u - d}.$$
- Mean matching:
$$\mathbb{E}[S_T \mid S_0] = S_0 e^{rT}.$$
- Variance matching:
$$u = \frac{b + \sqrt{b^2 - 4}}{2e^{r\Delta t}}, \quad d = \frac{1}{u}, \quad \text{where} \quad b = e^{(2r + \sigma^2)\Delta t} + 1.$$
- As $N \to \infty$, S_T converges to a log-normal distribution.

Lecture 3

What's the Next Step?

- European calls/puts and digitals are not the only tradable products. Other products of interest include European options with generic payoff, American options, Barrier options. etc.
- Many products have no closed-form solutions, and Binomial tree pricer can handle these cases.
- Goal: refine implementation to be easier to maintain and extend.

American Options

- At each node, compute the continuation value via conditional expectation.
- American options allow early exercise at any node.
- Two quantities at each node:
  - Intrinsic value:
$$\max(S-K,0)\ \text{(Call)}, \qquad \max(K-S,0)\ \text{(Put)}.$$
  - Continuation value: expected discounted future payoff.
- Decision rule:
$$V(t) = \max(\text{Intrinsic Value},\ \text{Continuation Value}).$$

Optimal Exercise Strategy

- Continue holding if continuation value exceeds intrinsic value.
- Exercise immediately if intrinsic value exceeds continuation value.
- At every node, maximize the option value.

Refining Code Structure

- Modify European pricer `n_step_binomial_tree`.
- At each iteration: continuation value (no exercise), payoff value (immediate exercise)
- Avoid copy–paste; improve extensibility.
- Allow flexible payoff definitions.

First-Step Extension

- Encapsulate tradable instrument parameters.
- European options:
  - expiry T, strike K, payoff type
- American options:
  - European features
  - early exercise capability

Tree Algorithm (American Option)

- Initialize terminal payoff.
- Backward induction for $k = N-1, \dots, 0$:
  - Compute discounted continuation value.
  - Compute option value from payoff and continuation.
- Step 5: tree pricer.
- Step 6: product-specific payoff rule.

Call Spread

- Strategy: buy and sell calls with different strikes.
- Bull call spread:
  - Buy call at lower strike K_{low}.
  - Sell call at higher strike K_{high}.
- Risk: limited to total premium paid.
- Reward: capped at $K_{\text{high}} - K_{\text{low}}$ minus cost.

Barrier Options

- Payoff depends on whether the underlying hits a barrier.
- Defined by:
  - one or two barriers (up/down),
  - monitoring window (usually $[0,T]$).
- Knock-In (KI): option becomes active only if barrier is hit.
- Knock-Out (KO): option expires if barrier is hit.
- Underlying payoff:
  - European payoff, or
  - fixed rebate (touch option).

Barrier Options Example (1): Up-and-In Call

- Strike price: $K = 100$
- Barrier level: $B = 120$
- Option premium: \$6

Payoff Scenarios

- Barrier touched, $S_T = 130$:
$$\text{Payoff} = 30, \quad \text{Net Profit} = 24$$
- Barrier not touched, $S_T = 130$:
$$\text{Payoff} = 0, \quad \text{Net Profit} = -6$$
- Barrier touched, $S_T = 95$:
$$\text{Payoff} = 0, \quad \text{Net Profit} = -6$$

Motivation

- Lower premium than vanilla call due to conditional activation.
- Useful when exposure is desired only after breaching a price level.

Barrier Options Example (2): Up-and-Out Call

- Strike price: $K = 100$
- Barrier level: $B = 120$
- Option premium: \$8

Payoff Outcomes

- Barrier not touched, $S_T = 110$:
$$\text{Payoff} = 10, \quad \text{Net Profit} = 2$$
- Barrier touched at any time:
$$\text{Payoff} = 0, \quad \text{Net Profit} = -8$$
- $S_T < 100$:
$$\text{Payoff} = 0, \quad \text{Net Profit} = -8$$

Motivation

- Cheaper than vanilla call since payoff is eliminated if barrier is hit.
- Suitable for moderate upside views below the barrier.

CRR Binomial Tree Recap

- Constraint: $u = \frac{1}{d}$.
- Second moment matching:
$$e^{(2r+\sigma^2)\Delta t} = pu^2 + (1-p)d^2.$$
- First moment matching:
$$p = \frac{e^{r\Delta t} - d}{u - d}.$$

Pricing Knock-Out Barrier Options

- At each node, check barrier condition.
- If barrier is triggered:
$$V = 0.$$
- Otherwise, continuation value:
$$V = e^{-r\Delta t}(pV_{\text{up}} + (1-p)V_{\text{down}}).$$
- Terminal payoff (if not knocked out):
$$\max(S-K,0)\ \text{(Call)}, \qquad \max(K-S,0)\ \text{(Put)}.$$

Alternative Binomial Tree Models

Jarrow–Rudd Model

- Equal probabilities (not risk-neutral): $p = \frac{1}{2}$.
- Parameters:
$$u = e^{(r-\frac{1}{2}\sigma^2)\Delta t + \sigma\sqrt{\Delta t}}, \quad d = e^{(r-\frac{1}{2}\sigma^2)\Delta t - \sigma\sqrt{\Delta t}}.$$

Tian Binomial Model

- Matches first three moments.
- Parameters:
$$p = \frac{e^{r\Delta t} - d}{u - d},$$
$$u = e^{(r-\frac{1}{2}\sigma^2)\Delta t + \sigma\sqrt{\Delta t}}, \quad d = e^{(r-\frac{1}{2}\sigma^2)\Delta t - \sigma\sqrt{\Delta t}}.$$

Options with Continuous Dividend Yield

- Continuous dividend yield at rate q:
$$1 \text{ unit at } t=0 \ \longrightarrow\ e^{qT} \text{ units at } T.$$
- Replicating portfolio at T:
$$\delta S_T e^{qT} + e^{rT}(V_0 - \delta S_0).$$
- Replication equations:
$$\begin{cases} \delta S_u e^{qT} + e^{rT}(V_0 - \delta S_0) = V_u, \\ \delta S_d e^{qT} + e^{rT}(V_0 - \delta S_0) = V_d. \end{cases}$$
- Solution:
$$\delta = e^{-qT}\frac{V_u - V_d}{S_u - S_d}, \qquad V_0 = e^{-rT}(pV_u + (1-p)V_d),$$
$$p = \frac{S_0 e^{(r-q)T} - S_d}{S_u - S_d}.$$

Black–Scholes and CRR Tree Models

- Without dividends:
$$\frac{dS_t}{S_t} = r\,dt + \sigma\,dW_t.$$
- With continuous dividend yield q:
$$\frac{dS_t}{S_t} = (r-q)\,dt + \sigma\,dW_t.$$
- Second moment matching:
$$\mathbb{E}[S_T^2] = e^{2(r-q)T + \sigma^2 T} = pu^2 + (1-p)d^2.$$
- CRR constraints ($u = 1/d$):
$$u = \frac{b + \sqrt{b^2-4}}{2}, \quad d = \frac{b - \sqrt{b^2-4}}{2},$$
$$b = e^{(r-q)T + \sigma^2 T} + e^{-(r-q)T}.$$

Path-Dependent Options in Trees

- American options: assume not exercised prior to current node.
- Barrier Knock-Out (KO): assume not knocked out yet.
- Other examples:
  - Knock-In (KI): activates only if barrier is hit,
  - Asian options: depend on average price history.

Barrier Options: KO vs KI

- **KO**: option becomes worthless if barrier is breached.
- **KI**: option becomes active only after barrier is breached.
- KO tree: track validity only.
- KI tree: track two states:
  - Barrier not triggered,
  - Barrier already triggered.

KIKO Parity

- KI and KO exhaust all barrier scenarios.
- Pricing identity:
$$\text{KI} = \text{Underlying payoff} - \text{KO}.$$
- Ensures arbitrage-free pricing and numerical stability.

Pricing Knock-In Options on Trees

- Two states at node (i,j):
$$V_{i,j}[0] \text{ (not triggered)}, \quad V_{i,j}[1] \text{ (triggered)}.$$
- Transition:
$$V_{i,j}[0] = \begin{cases} pV_{i+1,j}[0] + (1-p)V_{i+1,j+1}[0], & \text{no hit}, \\ pV_{i+1,j}[1] + (1-p)V_{i+1,j+1}[1], & \text{hit}, \end{cases}$$
$$V_{i,j}[1] = pV_{i+1,j}[1] + (1-p)V_{i+1,j+1}[1].$$

ValueAtNode Function

- Node value represented as a vector:
$$V_{i,j} = \begin{pmatrix} V_{i,j}[0] \\ V_{i,j}[1] \end{pmatrix} = f(\mathbb{E}^{\mathbb{Q}}[V_{i+1,j}]).$$
- Same logic as standard trees, but with vector-valued continuation.

Asian Options with Trees

- Payoff depends on average price $\bar{S}$:
$$\bar{S} = \frac{1}{n}\sum_{i=1}^{n} S_{T_i}.$$
- Asian call payoff:
$$\max(\bar{S} - K, 0).$$
- Decomposition at final fixing:
$$\bar{S} = \frac{1}{n}((n-1)A + S_{T_n}), \quad \tilde{K} = nK - (n-1)A.$$
- Auxiliary state: accumulated average A.
- Tree algorithm:
  - Discretize possible A values,
  - Backward induction for each A_i,
  - Node values returned as vectors.

Handling Intermediate and Fixing Dates (Asian Options)

Between Fixing Dates (T_{i-1}, T_i)

- The accumulated average A does not change,
- Node values equal continuation values.

At Fixing Dates T_i

The accumulated average is updated using the stock price S:
$$\widehat{A} = \frac{A \cdot i + S}{i+1}.$$

Interpolation of the Accumulated Average

Since $\widehat{A}$ may not lie on the grid $\{A_1, A_2, \ldots, A_m\}$, we apply linear interpolation.
Find j such that $\widehat{A} \in [A_j, A_{j+1}]$, and compute
$$V(\widehat{A}) = \frac{\widehat{A} - A_j}{A_{j+1} - A_j} V_{j+1} + \frac{A_{j+1} - \widehat{A}}{A_{j+1} - A_j} V_j,$$
where V_j, V_{j+1} are continuation values.

Additive Binomial Tree

- Use log-price as state variable: $X = \ln S$.
- Define increments:
$$u_x = \ln u, \qquad d_x = \ln d.$$

One-step dynamics (in X)
$$X_u = X_0 + \ln u = X_0 + u_x, \qquad X_d = X_0 + \ln d = X_0 + d_x,$$
with probabilities p and $1-p$ over one step Δt.

Calibration by matching moments

First moment

Black–Scholes solution for X:
$$X = X_0 + \left(r - \tfrac{1}{2}\sigma^2\right)\Delta t + \sigma W_{\Delta t}.$$
$$\mathbb{E}[X] = X_0 + \left(r - \tfrac{1}{2}\sigma^2\right)\Delta t.$$
In the binomial model,
$$\mathbb{E}[X] = X_0 + pu_x + (1-p)d_x = X_0 + \left(r - \tfrac{1}{2}\sigma^2\right)\Delta t. \tag{10}$$
Equivalently,
$$pu_x + (1-p)d_x = \left(r - \tfrac{1}{2}\sigma^2\right)\Delta t \implies p = \frac{\left(r - \frac{1}{2}\sigma^2\right)\Delta t - d_x}{u_x - d_x}.$$

Second moment
$$\mathbb{E}[X^2] = p(X_0 + u_x)^2 + (1-p)(X_0 + d_x)^2$$
Expanding:
$$\mathbb{E}[X^2] = X_0^2 + 2pX_0u_x + 2(1-p)X_0d_x + pu_x^2 + (1-p)d_x^2$$
Group terms:
$$\mathbb{E}[X^2] = X_0^2 + 2X_0(pu_x + (1-p)d_x) + pu_x^2 + (1-p)d_x^2. \tag{15}$$
Black–Scholes second moment:
$$X_0^2 + 2X_0\left(r - \tfrac{1}{2}\sigma^2\right)\Delta t + \left(r - \tfrac{1}{2}\sigma^2\right)^2\Delta t^2 + \sigma^2\,\mathbb{E}[W_{\Delta t}^2], \tag{16}$$
with
$$\mathbb{E}[W_{\Delta t}^2] = \Delta t.$$
Matching yields:
$$pu_x^2 + (1-p)d_x^2 - \left(r - \tfrac{1}{2}\sigma^2\right)^2\Delta t^2 = \sigma^2\Delta t. \tag{17}$$

CRR assumption
$$u_x = -d_x.$$

Convenient parameterization

Let
$$u_x = -d_x = \Delta x.$$
By CRR and (17):
$$\Delta x^2 = \sigma^2\Delta t + \left((r - \tfrac{1}{2}\sigma^2)\Delta t\right)^2,$$
so
$$\Delta x = \sqrt{\sigma^2\Delta t + \left((r - \tfrac{1}{2}\sigma^2)\Delta t\right)^2} \approx \sigma\sqrt{\Delta t} \qquad \text{when } \Delta t \to 0.$$
Also,
$$p = \frac{\left(r - \frac{1}{2}\sigma^2\right)\Delta t - d_x}{u_x - d_x} = \frac{1}{2} + \frac{r - \sigma^2/2}{2\sigma}\sqrt{\Delta t}.$$

Multi-Dimensional Binomial Method

Spread Option

A spread option gives payoff
$$[S_1(T) - S_2(T)]_+.$$
Assume both assets follow Black–Scholes with correlated Brownian motions:
$$\frac{dS_1}{S_1} = (r - q_1)dt + \sigma_1\,dW_1 \qquad \frac{dS_2}{S_2} = (r - q_2)dt + \sigma_2\,dW_2$$
and correlation
$$\rho\,dt = dW_1\,dW_2. \tag{24}$$

Log-state dynamics

Using the additive tree, define
$$x_1 = \ln S_1, \qquad x_2 = \ln S_2.$$
By Itô's lemma:
$$\begin{cases} dx_1 = \left(r - q_1 - \frac{1}{2}\sigma_1^2\right)dt + \sigma_1\,dW_1, \\ dx_2 = \left(r - q_2 - \frac{1}{2}\sigma_2^2\right)dt + \sigma_2\,dW_2. \end{cases} \tag{25}$$

Four-branch step (equal up/down jump sizes)

Choose equal up and down jump sizes:
$$\Delta x_1 = u_{x_1}, \qquad \Delta x_2 = u_{x_2}.$$
Each step has four outcomes with probabilities $p_{uu}, p_{ud}, p_{du}, p_{dd}$:
$$\begin{aligned} p_{uu}&: \quad (x_1 + \Delta x_1,\ x_2 + \Delta x_2), \\ p_{ud}&: \quad (x_1 + \Delta x_1,\ x_2 - \Delta x_2), \\ p_{du}&: \quad (x_1 - \Delta x_1,\ x_2 + \Delta x_2), \\ p_{dd}&: \quad (x_1 - \Delta x_1,\ x_2 - \Delta x_2). \end{aligned}$$

Calibration

Denote
$$v_1 = r - q_1 - \tfrac{1}{2}\sigma_1^2, \qquad v_2 = r - q_2 - \tfrac{1}{2}\sigma_2^2.$$
Then
$$\begin{cases} dx_1 = v_1\,dt + \sigma_1\,dW_1, \\ dx_2 = v_2\,dt + \sigma_2\,dW_2. \end{cases}$$
So
$$\mathbb{E}[dx_1] = v_1\Delta t, \qquad \mathbb{E}[dx_2] = v_2\Delta t,$$
$$\mathrm{Var}[dx_1] = \sigma_1^2\Delta t, \qquad \mathrm{Var}[dx_2] = \sigma_2^2\Delta t, \qquad \mathrm{Cov}[dx_1, dx_2] = \rho\sigma_1\sigma_2\Delta t.$$
In the binomial model,
$$\mathbb{E}[dx_1] = (p_{uu} + p_{ud})\Delta x_1 - (p_{du} + p_{dd})\Delta x_1,$$
$$\mathrm{Var}[dx_1] = (p_{uu} + p_{ud})\Delta x_1^2 + (p_{du} + p_{dd})\Delta x_1^2 - \left(\mathbb{E}[dx_1]\right)^2 \approx (p_{uu} + p_{ud})\Delta x_1^2 + (p_{du} + p_{dd})\Delta x_1^2 \quad (\Delta t \text{ is very small}),$$
$$\mathrm{Cov}[dx_1, dx_2] = (p_{uu} - p_{ud} - p_{du} + p_{dd})\Delta x_1\Delta x_2.$$

Matching system

$$\begin{cases} \mathbb{E}[dx_1] = (p_{uu} + p_{ud})\Delta x_1 - (p_{du} + p_{dd})\Delta x_1 = v_1 \Delta t, \\ \mathbb{E}[dx_2] = (p_{uu} + p_{du})\Delta x_2 - (p_{ud} + p_{dd})\Delta x_2 = v_2 \Delta t, \\ \mathrm{Var}[dx_1] = (p_{uu} + p_{ud})\Delta x_1^2 + (p_{du} + p_{dd})\Delta x_1^2 = \sigma_1^2 \Delta t + (v_1 \Delta t)^2 \approx \sigma_1^2 \Delta t, \\ \mathrm{Var}[dx_2] = (p_{uu} + p_{du})\Delta x_2^2 + (p_{ud} + p_{dd})\Delta x_2^2 = \sigma_2^2 \Delta t + (v_2 \Delta t)^2 \approx \sigma_2^2 \Delta t, \\ (p_{uu} - p_{ud} - p_{du} + p_{dd})\Delta x_1 \Delta x_2 = \rho \sigma_1 \sigma_2 \Delta t, \\ p_{uu} + p_{ud} + p_{du} + p_{dd} = 1. \end{cases}$$

Solution (model parameters)

$$\Delta x_1 = \sigma_1 \sqrt{\Delta t}, \qquad \Delta x_2 = \sigma_2 \sqrt{\Delta t},$$

$$\begin{cases} p_{uu} = \dfrac{1}{4} \dfrac{\Delta x_1 \Delta x_2 + \Delta x_2 v_1 \Delta t + \Delta x_1 v_2 \Delta t + \rho \sigma_1 \sigma_2 \Delta t}{\Delta x_1 \Delta x_2}, \\ p_{ud} = \dfrac{1}{4} \dfrac{\Delta x_1 \Delta x_2 + \Delta x_2 v_1 \Delta t - \Delta x_1 v_2 \Delta t - \rho \sigma_1 \sigma_2 \Delta t}{\Delta x_1 \Delta x_2}, \\ p_{du} = \dfrac{1}{4} \dfrac{\Delta x_1 \Delta x_2 - \Delta x_2 v_1 \Delta t + \Delta x_1 v_2 \Delta t - \rho \sigma_1 \sigma_2 \Delta t}{\Delta x_1 \Delta x_2}, \\ p_{dd} = \dfrac{1}{4} \dfrac{\Delta x_1 \Delta x_2 - \Delta x_2 v_1 \Delta t - \Delta x_1 v_2 \Delta t + \rho \sigma_1 \sigma_2 \Delta t}{\Delta x_1 \Delta x_2}. \end{cases}$$

Multi-dimensional binomial tree: node definition and pricing
For each node (k, i, j):
- k: time step (starting at $t = 0$),
- i: number of downward moves of asset 1 up to time k,
- j: number of downward moves of asset 2 up to time k.

For asset 1, upward moves are $k - i$, so cumulative change:

$$(k - i)\Delta x_1 - i\Delta x_1 = (k - 2i)\Delta x_1,$$

and similarly for asset 2: $(k - 2j)\Delta x_2$.
Thus prices at node (k, i, j) are

$$S_{1,k,i,j} = S_1 e^{(k-2i)\Delta x_1}, \qquad S_{2,k,i,j} = S_2 e^{(k-2j)\Delta x_2}.$$

Terminal payoff:

$$V = \max(0, S_1 - S_2).$$

Backward induction:

$$V_{k,i,j} = e^{-r\Delta t}\left[p_{uu} V_{k+1,i,j} + p_{ud} V_{k+1,i,j+1} + p_{du} V_{k+1,i+1,j} + p_{dd} V_{k+1,i+1,j+1}\right].$$

Analytic solution for spread option (Margrabe formula)
Price via change of numeraire; express payoff in units of S_2:

$$\left[\frac{S_1(T)}{S_2(T)} - 1\right]_+.$$

Under the dollar measure, the volatility of $\frac{S_1(T)}{S_2(T)}$ is

$$\sigma = \sqrt{\sigma_1^2 + \sigma_2^2 - 2\rho\sigma_1\sigma_2}.$$

Trinomial Tree Model
- The trinomial tree is an extension of the binomial tree model.
- At each time step, the underlying asset has three possible movements:
  - Upward movement: price multiplies by $u > 1$ with probability p_u,
  - Middle movement: price remains unchanged with probability p_m,
  - Downward movement: price multiplies by $d < 1$ with probability p_d.
- Probabilities satisfy:

$$p_u + p_m + p_d = 1.$$

Calibration of the Trinomial Tree
To calibrate parameters (u, d, p_u, p_m, p_d), we match moments of the discrete model to the continuous-time model.
- First moment (mean):

$$e^{\mu\Delta t} = p_u u + p_m + p_d d.$$

- Second moment:

$$e^{2\mu\Delta t + \sigma^2 \Delta t} = p_u u^2 + p_m + p_d d^2.$$

- Symmetry assumption:

$$u = \frac{1}{d}.$$

Underdetermined System
- There are 5 unknowns and only 4 equations.
- This leads to infinitely many solutions.
- To resolve this, introduce a tunable parameter λ controlling jump size.

Choice of Jump Sizes

$$u = e^{\lambda\sigma\sqrt{\Delta t}}, \qquad d = e^{-\lambda\sigma\sqrt{\Delta t}}.$$

Resulting Probabilities

$$p_u = \frac{1}{2\lambda^2} + \frac{\mu - \frac{1}{2}\sigma^2}{2\lambda\sigma}\sqrt{\Delta t},$$
$$p_d = \frac{1}{2\lambda^2} - \frac{\mu - \frac{1}{2}\sigma^2}{2\lambda\sigma}\sqrt{\Delta t},$$
$$p_m = 1 - p_u - p_d.$$

Example Choice
- Let $\lambda = \sqrt{3}$. Then:

$$u = e^{\sqrt{3\Delta t}}, \qquad d = e^{-\sqrt{3\Delta t}}.$$

- Corresponding probabilities:

$$p_u = \frac{1}{6} + \frac{\mu - \frac{1}{2}\sigma^2}{\sqrt{12}\sigma}\sqrt{\Delta t}, \quad p_d = \frac{1}{6} - \frac{\mu - \frac{1}{2}\sigma^2}{\sqrt{12}\sigma}\sqrt{\Delta t}, \quad p_m = \frac{2}{3}.$$

Remarks
- Trinomial trees often provide faster convergence than binomial trees.
- As $\Delta t \to 0$, the trinomial tree converges to the Black–Scholes model.

Lecture 4

From Constant to Implied Volatility
- **Black–Scholes assumption: constant volatility**

$$\frac{dS}{S} = (r - q)\,dt + \sigma\,dW$$

  - r: risk-free rate, q: dividend yield, σ: constant volatility.
  - This simplifies the mathematics and allows for a closed-form solution.
  - The volatility parameter σ must be specified by the user.
- **Market reality: volatility is not constant**
  - Volatility changes over time (volatility clustering).
  - Options with different strikes and maturities trade at different volatilities.
- **Implied volatility (IV): reverse of Black–Scholes**
  - Observe the market price of the option.
  - Solve for the σ that makes the Black–Scholes price match the market price.

Implied Volatility: Definition and Properties
- IV is obtained by **inverting** an option pricing model (e.g., Black–Scholes):
  - Input: observed market option price
  - Output: volatility that reproduces this price in the model
- **Key properties**
  - IV is model-derived, not directly observed.
  - IV is forward-looking (unlike historical volatility).
  - IV is not constant in practice.
- IV varies across:
  - Strike (moneyness), Maturity, Time (sentiment and events).
- **Interpretation:** market replaces constant-σ with IV to better reflect expectations.

Implied Volatility: Practical Meaning and Uses
- **Link to Black–Scholes**
  - Black–Scholes gives option price as a function of volatility.
  - Market practice reverses this relation to obtain implied volatility.
  - IV is the volatility input that makes the BS price match the market price.
  - Hence BS is a quoting and calibration tool, not only a pricing formula.
- **Why practitioners work with IV instead of price**
  - Option prices move mechanically with the spot price.
  - IV is more stable and comparable across contracts.
  - IV provides a normalized scale across strikes and maturities.
- **How traders use IV**
  - Evaluate whether an option is relatively expensive/cheap (vs. model).
  - Trade volatility directly (low-IV buy / high-IV sell strategies).
  - Compute Greeks and manage hedging risk consistently.
- **Result:** options are often quoted and managed in implied-volatility space.

Low vs. High Implied Volatility
- Options with higher IV are priced higher due to greater uncertainty.

Implied Volatility Surface
- In practice, vanilla options (standard calls/puts) are highly liquid and actively traded.
- Market participants agree on prices for common maturities and strikes, forming an **implied volatility surface**.
- Market makers must handle requests across a wide range of strikes and maturities.
- Directly managing option prices is impractical because:
  - Prices depend dynamically on the spot price, which fluctuates in real time.
  - Instead, participants quote IV, e.g. "105 call, 3-month, at 32% vol" rather than "price = 2.73".
- The IV surface allows traders to:
  - Interpolate/extrapolate prices for non-standard options,
  - Adjust for market conditions without recalibrating the entire pricing model.

Aspect	Low IV	High IV
Market expectation	Small moves	Large moves
Sentiment	Calm / Bullish	Fearful / Bearish
Risk	Lower	Higher
Option price	Lower	Higher

Black–Scholes Model: A Handy Tool for Option Pricing

- Directly managing option prices is challenging due to dependence on the underlying spot price.
- Black–Scholes provides a mathematical framework to compute option prices from key parameters:

$$V_{\text{call}}(K, S, \sigma, T, r) = S\,N(d_+) - e^{-rT} K\,N(d_-)$$

where

$$d_\pm = \frac{\ln\left(\frac{S}{K}\right) + (r-q)T \pm \frac{1}{2}\sigma^2 T}{\sigma\sqrt{T}}.$$

- with S current asset price, K strike, σ volatility, T time to maturity, r risk-free rate, q dividend yield, and $N(\cdot)$ the standard normal CDF.

Implied Volatility as an Implicit Function

- View Black–Scholes as a function from volatility σ to call price V:

$$f_{\{S,r,q,K,T\}}(\sigma) = V.$$

- Implied volatility is obtained by inverting this function:

$$\sigma = f^{-1}(V).$$

- **Key property:** the call price is monotone in σ:
  - When σ increases, V increases (higher volatility raises time value).
  - This holds as long as $T \geq 0$ (time remaining before expiration).
- However, f^{-1} has no closed form $\Rightarrow$ IV is an **implicit** function of σ.
- We therefore resort to a **root finding algorithm** to back out σ from market prices.

Root Finding: Solving $f(x) = 0$

- Given a function f, the goal is to determine the value(s) of x such that

$$f(x) = 0.$$

- This can arise in one-dimensional or multi-dimensional (N-dimensional) settings.
- **One-dimensional problem** ($f : \mathbb{R} \to \mathbb{R}$)
  - Easier to solve due to well-developed robust numerical algorithms.
  - Typically requires only function evaluations or derivatives to iteratively converge.
- **N-dimensional problem** ($f : \mathbb{R}^N \to \mathbb{R}$)
  - More challenging due to complex function landscapes.
  - Structure, curvature, and sensitivity often matter for efficient convergence.
- In implied volatility computation, we mainly deal with the **one-dimensional** case.

Bracketing the root: Ensuring the root is contained

Most fast root finding algorithms require an initial interval (bracket) that contains the root.
A bracket is an interval $[a, b]$ where $f(a)$ and $f(b)$ have opposite signs:

$$f(a) \cdot f(b) \leq 0.$$

This ensures that a root exists within the interval, provided that f is continuous.

How to Bracket the Root?

- Start with an initial interval that is likely to contain the root.
- If the initial interval does not satisfy $f(a) \cdot f(b) \leq 0$, adjust it by:
  - Expanding outward to search for a sign change,
  - Shrinking inward, if the function exhibits a known behavior in a smaller range.
- Once a valid bracket is found, iterative methods can be applied to refine the root estimate.

Root Bracketing Algorithm

Algorithm 1: RootBracketing(f, a, b, maxIter, factor)

- For $k = 1$ to maxIter:
- If $f(a)f(b) < 0$, return (a, b).
- If $|f(a)| < |f(b)|$, update

$$a \leftarrow a + \text{factor}\,(a - b).$$

- Else update

$$b \leftarrow b + \text{factor}\,(b - a).$$

If factor > 0, the algorithm searches outward.
If $-1 <$ factor < 0, the algorithm searches inward.

Root finding algorithms

Now we are equipped with a root bracketing algorithm.

Root finding

Given a function f, find the x such that

$$f(x) = 0.$$

Commonly used root finding algorithms:

- Bisection method
- Secant method
- False position method
- Brent's method
- Newton method

Bisection Method

Start with $[a, b]$ that brackets the root, cut it by half, then see which half contains the root.

Algorithm 2: rfbisect(f, a, b, tol)
Require: $f(a)f(b) < 0$, f is continuous, tolerance tol.

- while $(b - a)/2 >$ tol do
- $c \leftarrow \dfrac{a+b}{2}$
- if $f(c) = 0$ then return c
- else if $f(a)f(c) < 0$ then $b \leftarrow c$
- else $a \leftarrow c$

Bisection Method: Pro and Con

Pros

- Simple and easy to implement
- Only requires function evaluations, no derivatives needed
- Theoretically guaranteed to converge
- Robustness: if the interval contains more than one root, bisection will find one of them

Cons

- Slow convergence with only linear rate

$$\epsilon_n = |c_n - r| \leq \frac{b_n - a_n}{2}, \qquad c_n = \frac{a_n + b_n}{2}$$

Considering worst case:

$$\epsilon_{n+1} = \frac{1}{2}\epsilon_n.$$

Secant method

The secant method is motivated by the need for a faster alternative to bisection by utilizing the slope information of the function, yet does not require derivatives like Newton's method.
Consider a first-order Taylor expansion around a root x:

$$f(x) \approx f(x_n) + f'(x_n)(x - x_n).$$

Since $f(x) = 0$, we have

$$0 \approx f(x_n) + f'(x_n)(x - x_n) \quad \Rightarrow \quad x \approx x_n - \frac{f(x_n)}{f'(x_n)}.$$

Replace the derivative by the first-order difference:

$$f'(x_n) \approx \frac{f(x_n) - f(x_{n-1})}{x_n - x_{n-1}}.$$

Secant Method Iteration

The iterative scheme is:

$$x_{n+1} = x_n - f(x_n)\frac{x_n - x_{n-1}}{f(x_n) - f(x_{n-1})}.$$

Algorithm 3: rfsecant(f, x_1, x_2, tol)
Require: f is continuous, tolerance tol.

Secant Method: Pro and Con

Pros

- Open bracket method: does not require a bracket containing the root
- Faster convergence than bisection
- Superlinear convergence rate with order ≈ 1.62 (Golden Ratio)
- Works very well for nearly linear function

Cons

- Does not always converge, especially if:
  - The function has discontinuities or steep curvature
  - The denominator $f(x_{n+1}) - f(x_n)$ approaches zero
- Can fail without a proper stopping criterion
- Solution: implement a maximum iteration limit (maxIter)

False Position Method

The **false position method** (regula falsi) is a combination of the **bisection** and **secant** methods.

- Start with an interval $[a, b]$ that brackets the root.
- Draw a secant line through $(a, f(a))$ and $(b, f(b))$.
- Find its x-intercept at c.
- Update a or b to maintain the bracketed root.

Algorithm 4: `rffalsi`(f, a, b, tol)

- Require: $f(a)f(b) < 0$, f continuous
- While $b - a >$ tol:
- Compute

$$c \leftarrow \frac{af(b) - bf(a)}{f(b) - f(a)}.$$

- If $f(c) = 0$, return c.
- Else if $f(a)f(c) < 0$, set $b \leftarrow c$.
- Else set $a \leftarrow c$.

False Position Method: Pro and Con

Pros

- More robust than the secant method (maintains a bracket).
- Guaranteed to find a root if one exists.
- Often faster than bisection.

Cons

- Can converge slowly in certain cases.
- May behave similarly to bisection if one endpoint remains fixed.

Root Finding Algorithms: Considerations

Numerical methods are sensitive to:

- Initial values and assumptions,
- Problem characteristics,
- Tolerance settings,
- Round-off and truncation errors.

Before applying a root-finding algorithm, the problem must be analyzed carefully.

Brent's Method

The bisection method is robust but converges linearly. Secant and false-position methods converge faster but may fail. Brent's method is an adaptive hybrid strategy:

- Uses secant-like steps when they perform well,
- Falls back to bisection when needed,
- Ensures robustness while maintaining fast convergence.

Brent's method combines bisection, secant, inverse quadratic interpolation.

Inverse Quadratic Interpolation

This method improves convergence by fitting a quadratic function through the last three iteration points.

- Given three points
$$(x_0, f(x_0)),\ (x_1, f(x_1)),\ (x_2, f(x_2)),$$
fit
$$q(y) = ay^2 + by + c.$$
- Solve for the next estimate by taking
$$x = q(0).$$

Newton Method

Newton's method starts with one point instead of two.

The iteration scheme is:
$$x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)}.$$

At each iteration, it updates using the intercept of the tangent line with the x-axis.

Convergence Rate of Newton Method

Newton's method achieves **quadratic convergence** under suitable conditions.

Define
$$g(x) = x - \frac{f(x)}{f'(x)}.$$

Then Newton iteration is
$$x_{n+1} = g(x_n).$$

Define the error $e_n = x_n - r$. By Taylor expansion:
$$g(x_n) = g(r + e_n) = g(r) + g'(r)e_n + O(e_n^2).$$

Note that
$$g(r) = r, \qquad g'(r) = \frac{f(r)f''(r)}{f'(r)^2} = 0.$$

Hence
$$e_{n+1} = \frac{g''(\eta)}{2} e_n^2.$$

Newton Method: Potential Issues: Newton's method requires a good initial guess. Poor initialization may lead to divergence. The remedy is to mix Newton's method with bisection for stability.

Newton Method in Multi-Dimension Space

Newton's method can be extended to solve a system of non-linear equations:
$$f(x) = 0 \quad \to \quad \{f_i(x) = 0,\ 1 \le i \le N\}, \qquad x \in \mathbb{R}^N.$$

Update Rule in Multiple Dimensions

The update rule in multiple dimensions is given by:
$$x_{n+1} = x_n - J^{-1}(x_n)f(x_n).$$

- x_n is the current approximation of the root.
- $J(x)$ is the **Jacobian matrix**, containing first-order partial derivatives:
$$J_{ij} = \frac{\partial f_i}{\partial x_j}.$$

Computational Challenges

Evaluating and inverting $J(x)$ can be computationally expensive.

Instead of directly inverting $J(x)$, one typically solves the linear system:
$$J(x_n)\Delta x = -f(x_n), \qquad \Delta x = x_{n+1} - x_n.$$

Convergence Issues

Newton's method in high dimensions can face convergence difficulties due to:

- Complex nonlinear landscapes,
- Sensitivity to initialization,
- Expensive Jacobian computations.

Root Finding vs. Optimization in Quantitative Finance

In quantitative finance, we rarely encounter direct N-dimensional root-finding problems:
$$f(x) = 0, \qquad x \in \mathbb{R}^N.$$

This is because financial models typically do not require solving general nonlinear systems of equations without additional structure.

Optimization is More Common Instead, we frequently deal with:

- Minimization or maximization problems, where the goal is to optimize an objective function.
- Finding stationary points where the gradient satisfies:
$$\nabla g(x) = 0,$$

which is mathematically equivalent to solving a root-finding problem for the gradient. Financial models impose constraints on gradients, making them predictable and solvable with structured numerical techniques.

Lecture 5

Implied Volatility and Surface Construction

Converting Option Prices to Implied Volatility

We can now compute the implied volatility corresponding to a given option price.

In practice, options are often quoted using implied volatility instead of price because:

- implied volatility is less sensitive to changes than spot price,
- implied volatility is changing less frequently than spot price.

Discrete Data from Market Observations

Assume we have a discrete set of liquid maturities, such as 1W, 2W, 1M, 2M, 6M, 1Y, etc.

For each maturity (or pillar), market data provides a set of strike vs. option price tuples:
$$(k_i, C_i), \qquad i = 1, 2, \ldots, N.$$

Using the root-finding algorithm, we convert these into strike vs. implied volatility tuples:
$$(k_i, \sigma_i), \qquad i = 1, 2, \ldots, N.$$

Constructing the Implied Volatility Surface

To quote or price vanilla options at any arbitrary strike and maturity, we need to construct an implied volatility surface. It requires:

- **Interpolation**: estimating implied volatilities between known market points,
- **Extrapolation**: extending the volatility surface beyond observed strikes or maturities.

A well-constructed volatility surface allows traders and risk managers to:

- Price options consistently across different strikes and expirations,
- Identify market trends and arbitrage opportunities,
- Calibrate pricing models for hedging and risk management.

Volatility Smile Interpolation

Volatility Smile

The function $k \mapsto \sigma$ for a fixed maturity describes how implied volatility changes with strike price.

This function is often referred to as the **volatility smile** due to its typical convex shape.

Understanding the Pattern

Higher implied volatility for deep in-the-money (ITM) and out-of-the-money (OTM) strikes:

- Options far from the current stock price tend to have higher implied volatility.
- Deep ITM and OTM options have lower liquidity and are more sensitive to large price swings.
- Market participants demand higher volatility premiums to compensate for uncertainty.

Lower implied volatility near at-the-money (ATM):

- ATM options typically exhibit the lowest implied volatility.
- These options are the most liquid and actively traded.
- Their implied volatility is stabilized by market forces.

Starting with Linear Interpolation

Given sample points (x_i, y_i), $i \in [0, 1, \ldots, N]$, linear interpolation constructs a piecewise linear function $L(x)$ of the form
$$L(x) = L_i(x), \qquad x \in [x_i, x_{i+1}], \quad i = 0, \ldots, N-1.$$

Such that:

- Each L_i is a linear polynomial,
- $L_0(x_0) = y_0$, $L_{N-1}(x_N) = y_N$,
- $L_i(x_{i+1}) = L_{i+1}(x_{i+1}) = y_{i+1}$.

The linear function is uniquely determined by:
$$L_i(x_i) = y_i, \qquad L_i(x_{i+1}) = y_{i+1}.$$

Let $h_i = x_{i+1} - x_i$, then for $x \in [x_i, x_{i+1}]$:
$$L_i(x) = y_{i+1}\frac{x - x_i}{h_i} + y_i\frac{x_{i+1} - x}{h_i}.$$

Cubic Spline Interpolation

Given sample points (x_i, y_i), cubic spline interpolation constructs a C^2 piecewise cubic polynomial $S(x)$ of the form
$$S(x) = S_i(x), \qquad x \in [x_i, x_{i+1}], \quad i = 0, \ldots, N-1.$$

Such that:

- Each S_i is a cubic polynomial,
- $S_0(x_0) = y_0$, $S_{N-1}(x_N) = y_N$,
- $S_i(x_{i+1}) = S_{i+1}(x_{i+1}) = y_{i+1}$,
- $S_i'(x_{i+1}) = S_{i+1}'(x_{i+1})$,
- $S_i''(x_{i+1}) = S_{i+1}''(x_{i+1})$.

Assume the second derivatives at each node are:
$$S''(x_j) = M_j, \qquad 0 \le j \le N.$$

Then for each S_i:
$$S_i(x_i) = y_i, \quad S_i(x_{i+1}) = y_{i+1}, \quad S_i''(x_i) = M_i, \quad S_i''(x_{i+1}) = M_{i+1}.$$

The four conditions determine a unique cubic polynomial. Let $h_i = x_{i+1} - x_i$:
$$S_i(x) = \frac{M_{i+1}}{6h_i}(x - x_i)^3 + \frac{M_i}{6h_i}(x_{i+1} - x)^3 + \left(y_i - \frac{M_i h_i^2}{6}\right)\frac{x_{i+1} - x}{h_i} + \left(y_{i+1} - \frac{M_{i+1}h_i^2}{6}\right)\frac{x - x_i}{h_i}.$$

Recall S has continuous first derivatives:
$$S_i'(x_{i+1}) = S_{i+1}'(x_{i+1}).$$

Setting this yields the standard tridiagonal system:

$$h_{i-1}M_{i-1} + 2(h_{i-1} + h_i)M_i + h_i M_{i+1} = 6\left(\frac{y_{i+1} - y_i}{h_i} - \frac{y_i - y_{i-1}}{h_{i-1}}\right), \qquad 1 \le i \le N-2.$$

Additional boundary conditions:

- Natural spline:

$$M_0 = 0, \qquad M_{N+1} = 0.$$

- Or impose boundary values on first derivatives:

$$S_0'(x_0) = p_0, \qquad S_{N-1}'(x_N) = p_1.$$

Cubic Spline Interpolation in our Application

We may impose clamped boundary conditions by setting y_0' and y_{N-1}' to 0. This ensures flattened volatility beyond the boundary.

Interpolation can be applied to either variance σ^2 or volatility σ.

No-Arbitrage Considerations

- Interpolated implied volatilities are not guaranteed to be arbitrage-free.
- This can be problematic in pegged currency markets where strict rules prevent arbitrage.

Time Interpolation in Implied Volatility

At each maturity, we have a volatility smile capturing the variation of implied volatility across strikes.

When evaluating options expiring between pillar dates, we must interpolate implied volatility across time to obtain a smooth volatility surface.

A common choice is **linear interpolation on total variance** σ^2 along the strike line. This ensures that variance increases with time, which is necessary to preserve calendar arbitrage constraints.

- For options with the same strike, longer maturity implies higher option value.
- Interpolating variance guarantees variance remains increasing, provided variance at the next pillar is higher than at the previous one.

Mathematical Formulation

We interpolate the total variance linearly in time:

$$v_t(K) = \frac{t_{i+1} - t}{t_{i+1} - t_i} v_{t_i}(K) + \frac{t - t_i}{t_{i+1} - t_i} v_{t_{i+1}}(K), \qquad v_t(K) = \sigma_t^2(K)\, t.$$

Implementation of Implied Volatility Surface

1. Interpolation Across Strike (Fixed Maturity)

For each maturity t_i:

- Market provides implied volatility at discrete strikes.
- Construct a smooth volatility smile using **cubic spline interpolation**:

$$\sigma_{t_i}(K) = S_{t_i}(K).$$

Purpose: smooth variation across strike and avoid unrealistic jumps in volatility.

2. Interpolation Across Time (Fixed Strike)

For a given strike K, volatility must be interpolated across maturities. We interpolate **total variance**, not volatility:

$$v(t, K) = \sigma(t, K)^2\, t.$$

Reason: total variance should increase with maturity to prevent calendar arbitrage.

Summary: Computing Implied Volatility at Strike K and Maturity t

- Step 1: For each market maturity t_i, construct cubic spline $S_{t_i}(K)$.
- Step 2: Locate surrounding maturities $t_i \le t \le t_{i+1}$.
- Step 3: Evaluate spline at strike K:

$$\sigma_{t_i}(K) = S_{t_i}(K), \qquad \sigma_{t_{i+1}}(K) = S_{t_{i+1}}(K).$$

- Step 4: Convert to total variance:

$$v_{t_i}(K) = \sigma_{t_i}(K)^2 t_i, \qquad v_{t_{i+1}}(K) = \sigma_{t_{i+1}}(K)^2 t_{i+1}.$$

- Step 5: Linear interpolation in time:

$$v_t(K) = \frac{t_{i+1} - t}{t_{i+1} - t_i} v_{t_i}(K) + \frac{t - t_i}{t_{i+1} - t_i} v_{t_{i+1}}(K).$$

- Step 6: Convert back to volatility:

$$\sigma_t(K) = \sqrt{\frac{v_t(K)}{t}}.$$

Market Variations in Volatility Quotes

- Some asset classes (commodities, equities, rates) provide option price quotes across strikes and maturities.
- Other markets, such as FX options, do not directly quote a full volatility surface.

Since FX markets do not provide implied volatilities for all strikes, an extra step is needed to map market quotes into surface inputs.

What are FX Options?

FX options give the right (but not obligation) to exchange a specified amount of one currency for another at a pre-agreed strike rate on or before maturity.

Key Features of FX Options

- Two interest rates: domestic and foreign.
- Underlying asset: exchange rate (currency pair).
- Premium payment typically in domestic currency.

How Option is Quoted in FX Options

In FX options, traders often summarize the volatility smile by quoting:

- **ATM volatility**: implied volatility of an option whose strike equals the forward exchange rate.
- **Risk Reversal (RR)**: measures skew of the implied volatility smile.
- **Butterfly (BF)**: measures smile curvature (fat-tailedness).

Quoting via ATM, RR, BF provides a compact way to negotiate, price, and hedge FX options.

FX Market Conventions: Spot

- FX spot rate: exchange rate at which two currencies can be exchanged immediately.
- In practice, FX spot transactions settle in 1 or 2 days ($T+1$ or $T+2$).
- This delay is called the **spot-lag**.

For simplicity, we ignore the spot-lag in this course.

FX Market Conventions: Moneyness

In FX options, the strike price K is expressed as an exchange rate.

Moneyness is determined by comparing strike K to forward price F:

$$F = Se^{(r_d - r_f)T},$$

where S is spot price and T is maturity.

- At-the-Money (ATM): $K = F$.
- In-the-Money (ITM): Call $K < F$, Put $K > F$.
- Out-of-the-Money (OTM): Call $K > F$, Put $K < F$.

FX Market Conventions: ATMVOL

ATMVOL refers to the implied volatility of an ATM option in FX markets.

Two common definitions:

- ATM Forward (ATM-FWD)
- ATM Delta-Neutral Straddle (ATM-DNS)

ATM Forward strike is set to the forward price:

$$K_{ATM} = F = Se^{\int_0^t (r_d(s) - r_f(s))ds}.$$

Since strike matches forward price, call and put have the same price (put-call parity).

FX Market Conventions: Delta

Delta measures sensitivity of an option price to changes in the underlying asset.

- Call options: Delta ranges from 0 to $+1$.
- Put options: Delta ranges from 0 to -1.

Key principle: an option can never react more strongly than owning the asset itself.

Delta for Black-Scholes Price (CALL)

The Black–Scholes price for a European FX call option is:

$$Call_{BS} = Se^{-r_f T} N(d_+) - Ke^{-r_d T} N(d_-),$$

with

$$d_\pm = \frac{\ln(F/K) \pm \frac{1}{2}\sigma^2 T}{\sigma\sqrt{T}}.$$

where:

- r_f: foreign risk-free rate,
- r_d: domestic risk-free rate,
- $F = Se^{(r_d - r_f)T}$: forward exchange rate.

The call delta is:

$$\Delta_{call}(\sigma, K, T, r_d, r_f) = \frac{\partial Call_{BS}}{\partial S} = e^{-r_f T} N(d_+).$$

Delta (PUT)

The Black–Scholes price for a European FX put option is:

$$Put_{BS} = Ke^{-r_d T} N(-d_-) - Se^{-r_f T} N(-d_+).$$

The put delta is:

$$\Delta_{put}(\sigma, K, T, r_d, r_f) = \frac{\partial Put_{BS}}{\partial S} = -e^{-r_f T} N(-d_+).$$

Forward delta (derivative w.r.t. forward price F) is:

$$\Delta_{call} = N(d_+), \qquad \Delta_{put} = -N(-d_+).$$

ATM Delta-Neutral Straddle (ATM-DNS)

A straddle consists of:

- Buying one call and one put,
- Same strike K and maturity T.

The ATM-DNS strike is defined by zero net delta:

$$\Delta_{call}(K_{DNS}) + \Delta_{put}(K_{DNS}) = 0.$$

Solving the delta-neutral condition gives:

$$K_{DNS} = Fe^{\frac{1}{2}\sigma_{ATM}^2 T}.$$

FX Market Convention — Our Use Case

In practice, FX markets often use:

- ATM forward convention,
- Forward delta instead of spot delta.

Forward delta ranges:

$$\Delta_{call} = N(d_+), \qquad \Delta_{put} = -N(-d_+).$$

Using Delta
Delta is fundamental in FX options trading:
- First risk to hedge,
- Standardized measure of moneyness.

FX options are quoted by delta levels Δ, e.g.

$$25\Delta \text{ Call}.$$

The strike K must be found by solving:

$$0.25 = \Delta_{call} = N(d_+), \qquad d_+ = \frac{\ln(F/K) + \frac{1}{2}\sigma^2 T}{\sigma\sqrt{T}}.$$

Market Strangle
A market strangle is:
- Buy one OTM call (higher strike),
- Buy one OTM put (lower strike).

It hedges large price moves in either direction.
Market Strangle Volatility Spread
In FX, strangles are quoted as a volatility spread relative to ATM:

$$\sigma^{\delta}_{ms}.$$

The OTM volatilities satisfy:

$$\frac{\sigma_c + \sigma_p}{2} = \sigma_{ATM} + \sigma^{\delta}_{ms},$$

where:
- σ_c: implied vol of OTM call,
- σ_p: implied vol of OTM put,
- σ_{ATM}: ATM implied vol.

Pricing the Strangle
Under Black–Scholes:

$$\text{Strangle price} = C(K_c, \sigma_c) + P(K_p, \sigma_p).$$

Market strangles constrain the curvature of the volatility smile:
- Expensive strangles $\rightarrow$ stronger convexity,
- Cheap strangles $\rightarrow$ flatter smile.

The strikes are determined by delta conditions:

$$\Delta_{call}(K_c, \sigma_{ATM} + \sigma_{\Delta_{ms}}) = \Delta,$$

$$\Delta_{put}(K_p, \sigma_{ATM} + \sigma_{\Delta_{ms}}) = -\Delta.$$

Link Between Market Strangle and Volatility Smile
- In a market strangle, traders rely on implied volatility to generate profits.
- The implied volatility curve follows a volatility smile:
  - Out-of-the-money (OTM) options have higher implied volatility,
  - Hence OTM calls and puts are more expensive.
- If the volatility smile is steep, strangles become more costly.
- If the smile is flatter, strangles are cheaper but profit potential is smaller.

Market Role of Strangles
- Constrains the volatility smile and surface shape.
- Measures smile convexity:

$$\text{greater strangle spread} \;\Rightarrow\; \text{more convex smile}.$$

-
Smile Strangle and Risk Reversal
Smile Strangle (Butterfly)
The smile strangle BF_Δ (butterfly spread) measures convexity:

$$BF_\Delta = \frac{1}{2}\left(\sigma_{\Delta_c} + \sigma_{\Delta_p}\right) - \sigma_{ATM}.$$

- Δ_c: delta level of the call option
- Δ_p: delta level of the put option
- Example: 25Δ smile strangle averages implied vols of 25Δ call/put minus ATM.

Risk Reversal
The risk reversal RR_Δ measures asymmetry (skewness):

$$RR_\Delta = \sigma_{\Delta_c} - \sigma_{\Delta_p}.$$

- Shows difference between implied vol of OTM calls and OTM puts.
- If $RR_\Delta > 0$, calls are more expensive (bullish skew).
- If $RR_\Delta < 0$, puts are more expensive (bearish skew).

The FX Volatility Marking Problems
In FX markets, implied volatility is quoted at specific delta levels rather than fixed strikes.
Given ATM volatility σ_{ATM}, smile strangles, and risk reversals, we solve:

$$\begin{cases} BF_\Delta = \frac{1}{2}(\sigma_{\Delta_c} + \sigma_{\Delta_p}) - \sigma_{ATM}, \\ RR_\Delta = \sigma_{\Delta_c} - \sigma_{\Delta_p}. \end{cases}$$

This yields:

$$\sigma_{\Delta_c} = \sigma_{ATM} + BF_\Delta + \frac{1}{2}RR_\Delta, \qquad \sigma_{\Delta_p} = \sigma_{ATM} + BF_\Delta - \frac{1}{2}RR_\Delta.$$

Converting Delta Quotes to Strike-Based Inputs
Market quotes are given as (Δ_i, σ_i) pairs, but we need strike-volatility pairs (K_i, σ_i).
Convert using:

$$\Delta_{call}(K_c, \sigma) = \Delta, \qquad \Delta_{put}(K_p, \sigma) = -\Delta.$$

Then K_i is found by solving these equations via root-finding.
Market Data Considerations
- Risk reversal RR is directly quoted.
- Smile strangle BF is not explicitly quoted: market provides only *market strangle* data.
- Challenge: construct the full implied volatility surface from ATM, RR, and strangles.

Market Strangle, Risk Reversal, and Surface Construction
- Risk reversal is observable in the raw market.
- Smile strangle is not directly observable; only market strangles are given.
- Key steps in volatility surface construction:
  - Collect bid-ask spreads for strangles and risk reversals across maturities.
  - Interpolate missing smile strangle data using spline or arbitrage-free methods.
  - Apply advanced volatility models to fit observed quotes.
  - Ensure no-arbitrage consistency with risk-neutral pricing.

How About Diffusion Models?
So far, we built an implied volatility surface to match vanilla option quotes.
However, Black–Scholes assumes constant volatility and cannot reproduce the full smile across strikes and maturities.
Need for an Advanced Model
- We are model-less: no full stochastic process for the underlying.
- Exotic and path-dependent options require richer models.
- Any advanced model must calibrate to the implied volatility surface.

Local Volatility Model
The local volatility model is a natural extension of Black–Scholes:
- Volatility depends on both time and underlying price.
- Designed to fit the implied volatility surface exactly.
- Provides a powerful framework for pricing exotic options.

Lecture 6

Black-Scholes Model and Implied Volatility
- The Black-Scholes formula determines option prices based on a model that assumes constant volatility.
- However, market prices indicate that implied volatility varies across different strikes and maturities, leading to the concept of the implied volatility smile and implied volatility surface.
- This inconsistency suggests that we need refinements such as stochastic volatility models or local volatility models to better match observed market behavior.
- The implied volatility surface provides a more intuitive representation of market expectations, helping traders and analysts better assess option pricing dynamics.

Black-Scholes Term Structure Model
- Extension of Black-Scholes with time-dependent volatility:

$$\frac{dS}{S} = \mu(t)dt + \sigma(t)dW$$

- Distinction between instantaneous volatility $\sigma(t)$ and implied volatility $\sigma_{iv}(K,T)$ inferred from market prices
- $\sigma_{iv}(K,T)$ reflects market consensus across strikes K and maturities T.
- Local volatility $\sigma(S,t)$ directly drives the stochastic process.
- These coincide only when volatility is constant.
- Connection Through Total Variance

$$\int_0^T \sigma^2(u)\,du$$

$$\sigma^2_{iv}(T)T = \int_0^T \sigma^2(u)\,du$$

- Implied volatility summarizes cumulative instantaneous variance.
- Recovering $\sigma(S,t)$ from $\sigma_{iv}(K,T)$ is an inverse problem.
- Assume no volatility smile (no strike dependence), but ATM implied volatility varies with maturity.
- Market quotes $\sigma_{iv}(T_i)$ at discrete maturities.
- Piecewise-constant instantaneous volatility:

$$\sigma(t) = \sigma_i, \quad t \in [T_i, T_{i+1}]$$

$$\sigma^2_{iv}(T_{i+1})T_{i+1} = \sigma^2_{iv}(T_i)T_i + \sigma^2_i(T_{i+1} - T_i)$$

$$\sigma^2_i = \frac{\sigma^2_{ATM}(T_{i+1})T_{i+1} - \sigma^2_{ATM}(T_i)T_i}{T_{i+1} - T_i}$$

- Instantaneous variance equals slope of total variance.
- Guarantees match to ATM term structure.

Dependency on Strike Prices
- Strike dependence is more complex than maturity dependence.
- Goal: construct a diffusion consistent with full implied volatility surface.
- Approaches:
  - Stochastic volatility (Heston, SABR)
  - Jump diffusion
- Trade-offs:
  - Additional risk factors
  - Numerical complexity
  - Residual calibration errors

Local Volatility Model

$$\frac{dS}{S} = \mu(t)dt + \sigma(S,t)dW$$

- Volatility depends on both price and time.
- Exactly matches implied volatility surface.
- The way to compute $\sigma(S,t)$ and the uniqueness of solution is addressed by Dupire's Model.

Dupire's Equation: Extracting Local Volatility

$$\sigma^2(S,t) = \frac{\frac{\partial}{\partial T}\left(\sigma_{iv}^2 K^2 T\right)}{1 + K\frac{\partial}{\partial K}\left(\sigma_{iv}^2 K^2 T\right)}$$

- Requires differentiating the implied volatility surface.
- Numerator: time sensitivity of total variance.
- Denominator: strike sensitivity adjustment.
- Produces a local volatility surface consistent with market prices.

Local Volatility and Arbitrage

- Dupire formula:

$$\sigma^2(K,T) = \frac{2\,\frac{\partial C(K,T)}{\partial T}}{K^2\,\frac{\partial^2 C(K,T)}{\partial K^2}}$$

- Denominator(Breeden–Litzenberger result):

$$K^2\,\frac{\partial^2 C(K,T)}{\partial K^2} = K^2 p(K,T)$$

- Well-posedness requires:

$$p(K,T) \geq 0$$

No-Arbitrage Conditions on Call Prices

- Positivity:

$$C(K,T) \geq 0$$

- Monotonicity in strike:

$$\frac{\partial C}{\partial K} \leq 0$$

- Convexity in strike (butterfly arbitrage-free):

$$\frac{\partial^2 C}{\partial K^2} \geq 0$$

- Monotonicity in maturity (calendar arbitrage-free):

$$\frac{\partial C}{\partial T} \geq 0$$

Implications for Local Volatility

- Numerator $\partial C/\partial T$ must be non-negative.
- Denominator $K^2\partial^2 C/\partial K^2$ must be positive.
- Violations lead to:
  - Negative local variance
  - Explosive or undefined diffusion
- Hence: implied volatility surface must be arbitrage-free.

From Call Prices to Implied Volatility

- Market provides implied vol surface $\sigma_{iv}(K,T)$.
- Convert to call prices via Black–Scholes:

$$C(K,T) = BS(S_0, K, T, \sigma_{iv}(K,T))$$

- Differentiate numerically to compute:

$$\frac{\partial C}{\partial T}, \qquad \frac{\partial^2 C}{\partial K^2}$$

- Plug into Dupire formula.

Practical Implementation Steps

1. Construct arbitrage-free implied volatility surface.
2. Convert to smooth call price surface.
3. Compute derivatives w.r.t. K and T.
4. Apply Dupire formula.
5. Obtain local volatility surface $\sigma(S,t)$.

Numerical Challenges

- Differentiation amplifies noise.
- Sparse market data.
- Need stable interpolation across:
  - Strike dimension
  - Maturity dimension
- Often use:
  - Spline smoothing
  - Parametric volatility models
  - Arbitrage-free parameterizations

Economic Interpretation

- Local volatility matches today's smile exactly.
- It encodes market-implied distribution of future prices.
- However, future smile dynamics are deterministic.

Limitations of Local Volatility

- Does not reproduce realistic smile dynamics.
- Forward-start options often mispriced.
- Volatility-of-volatility effects absent.
- Motivates stochastic volatility extensions.

Big Picture

- Fokker–Planck links SDEs to density evolution.
- Breeden–Litzenberger links density to option prices.
- Dupire links option prices to local volatility.
- Full chain: Market Smile $\Rightarrow$ $C(K,T)$ $\Rightarrow$ $p(K,T)$ $\Rightarrow$ $\sigma(S,t)$

Convexity Condition of Call Prices

- For any no-arbitrage call price function, convexity in strike is required:

$$\frac{\partial^2 C(K,T)}{\partial K^2} = p(K,T) \geq 0$$

- If convexity is violated at some K:

$$p(K,T) = \frac{\partial^2 C(K,T)}{\partial K^2} < 0$$

- Negative density implies arbitrage.
- This is called strike (butterfly) arbitrage.
- In practice, non-convexity must be corrected.

How Local Volatility Handles Arbitrage Issues

- Local volatility:

$$\sigma(K,T) = \sqrt{\frac{2\,\frac{\partial C(K,T)}{\partial T}}{K^2\,\frac{\partial^2 C(K,T)}{\partial K^2}}}$$

- If $\frac{\partial^2 C}{\partial K^2} < 0$, σ becomes complex.
- Practical fix: floor the denominator ($\varepsilon > 0$ small threshold).

$$K^2 \max\left(\frac{\partial^2 C}{\partial K^2}, \varepsilon\right)$$

Role of the Numerator

- Numerator: $2\,\frac{\partial C(K,T)}{\partial T}$ (dual theta).
- Expected: $\frac{\partial C(K,T)}{\partial T} \geq 0$
- Violation $\Rightarrow$ calendar arbitrage.
- Numerator often floored: $2\max\left(\frac{\partial C(K,T)}{\partial T}, \varepsilon\right)$

Local Volatility – General Case (Non-zero Drift)

$$\sigma(K,T) = \sqrt{\frac{2\left[\frac{\partial C}{\partial T} + (r_d - r_f)K\frac{\partial C}{\partial K} + r_f C\right]}{K^2\,\frac{\partial^2 C}{\partial K^2}}}$$

- Only numerator changes vs driftless case.
- Depends on domestic r_d and foreign r_f rates (FX case).

Derivation Outline

- SDE:

$$dS_t = (r_d - r_f)S_t dt + \sigma(S_t,t)S_t dW_t$$

- Fokker-Planck equation:

$$\frac{\partial p}{\partial t} = -\frac{\partial}{\partial s}[(r_d - r_f)sp] + \frac{1}{2}\frac{\partial^2}{\partial s^2}[s^2\sigma^2 p]$$

- Using $p(K,T) = \frac{\partial^2 C}{\partial K^2}$,
- Integrate and solve for $\sigma(K,T)$.

Local Volatility in Terms of Implied Volatility

- Since FX markets quote implied vols, rewrite formula using $\sigma_{iv}(K,T)$.

$$\sigma(K,T) = \sqrt{\frac{\sigma_{iv}^2 + 2T\sigma_{iv}\frac{\partial\sigma_{iv}}{\partial T} + 2(r_d - r_f)KT\sigma_{iv}\frac{\partial\sigma_{iv}}{\partial K}}{\left(1 + Kd_1\sqrt{T}\frac{\partial\sigma_{iv}}{\partial K}\right)^2 + K^2 T\sigma_{iv}\left(\frac{\partial^2\sigma_{iv}}{\partial K^2} - d_1\sqrt{T}\left(\frac{\partial\sigma_{iv}}{\partial K}\right)^2\right)}}$$

$$d_1 = \frac{\ln(F/K) + \frac{1}{2}\sigma_{iv}^2 T}{\sigma_{iv}\sqrt{T}}$$

Finite Difference Approximation of Derivatives

First-order derivatives

$$f'(x) \approx \frac{f(x+h) - f(x)}{h} \qquad f'(x) \approx \frac{f(x) - f(x-h)}{h} \qquad f'(x) \approx \frac{f(x+h) - f(x-h)}{2h}$$

Second-order derivative

$$f''(x) \approx \frac{f(x+h) + f(x-h) - 2f(x)}{h^2}$$

- Large $h \Rightarrow$ truncation error.
- Small $h \Rightarrow$ round-off error.

What Does the Local Volatility Surface Look Like?

- Jumps at pillars mainly due to discontinuities of $\partial\sigma_{iv}/\partial T$.
- Smooth along strike due to C^2 spline interpolation.

Lecture 7

Martingality of Discounted Derivative Price

- Key idea: Risk-neutral pricing determines the fair value of derivatives.
- Under the risk-neutral measure $\mathbb{Q}$, the discounted derivative price is a martingale.
- This means the expected discounted future price equals the current discounted price.

$$\frac{V_t}{B_t} = \mathbb{E}_{\mathbb{Q}}\left[\frac{V_T}{B_T}\right]$$

- $\mathbb{Q}$ = risk-neutral probability measure.
- B_t = value of the risk-free investment (money market account).
- V_t = price of the derivative at time t.
- The result holds whether B_t is deterministic or stochastic.

Why Do We Discount B_t?

- If interest rates are constant:

$$B_t = e^{rt}$$

- Money today is worth more than money in the future due to interest rates.
- Discounting adjusts prices for the time value of money.

Martingale Property

- A martingale is a process whose conditional expectation equals its current value.
- Ensures absence of arbitrage opportunities.

Implication for Derivative Pricing

- Pricing derivatives reduces to computing expectations.
- The fair value equals the expected discounted payoff under $\mathbb{Q}$.

$$\frac{V_t}{B_t} = \mathbb{E}_{\mathbb{Q}}[h(X)]$$

- X represents the random factors affecting the payoff.
- $h(X)$ is the payoff function of the derivative.

Monte Carlo Simulation for Expectations

- Monte Carlo methods approximate expectations numerically.
- Useful when:
  - Analytical solutions do not exist.
  - The underlying process is complex.
  - The problem involves many random variables.
- Handles exotic and path-dependent derivatives.
- Scales well in high-dimensional problems.

Law of Large Numbers (LLN)

- Let $X_1, X_2, \ldots, X_n$ be i.i.d. random variables.

$$\mathbb{E}[X_i] = m, \qquad \mathrm{Var}(X_i) = \sigma^2 < \infty$$

- Consider

$$\mu = \mathbb{E}[h(X)]$$

- Define the sample mean estimator:

$$\hat{\mu}_n = \frac{1}{n}\sum_{i=1}^{n} h(X_i)$$

- LLN states:

$$P(|\hat{\mu}_n - \mu| \leq \epsilon) \to 1 \quad \text{as } n \to \infty$$

LLN and Monte Carlo

- The sample average approximates the true expectation.
- The estimator is unbiased:

$$\mathbb{E}[\hat{\mu}_n] = \frac{1}{n}\mathbb{E}\left[\sum_{i=1}^{n} h(X_i)\right] = \mu$$

- Monte Carlo procedure:
  - Generate many independent samples.
  - Compute the payoff for each sample.
  - Average the results.

Monte Carlo Algorithm

1. Initialize $s = 0$.
2. For $i = 1, \ldots, n$:
   - Generate X_i.
   - Compute $h_i = h(X_i)$.
   - Update $s = s + h_i$.
3. Return

$$\hat{\mu}_n = \frac{s}{n}$$

Monte Carlo as an Integration Method

- Many integrals can be written as expectations.

$$\int_0^1 f(x)\,dx = \mathbb{E}[f(x)], \quad x \sim U(0,1)$$

- For a general interval $[a, b]$:

$$\mathbb{E}[f(x)] = \int_a^b f(x)p(x)dx = \frac{1}{b-a}\int_a^b f(x)dx$$

$$\int_a^b f(x)dx = (b-a)\mathbb{E}[f(x)], \quad x \sim U(a,b)$$

- Monte Carlo approximation:

$$\int_a^b f(x)dx \approx (b-a)\frac{1}{N}\sum_{i=1}^{N} f(x_i)$$

- Works well for high-dimensional integrals.
- Effective when analytical integration is difficult.

Estimating Probabilities with Monte Carlo

- A probability can be expressed as an expectation:

$$P(X \in A) = \mathbb{E}[I_A(X)]$$

- Indicator function:

$$I_A(X) = \begin{cases} 1 & X \in A \\ 0 & \text{otherwise} \end{cases}$$

- Monte Carlo estimator:

$$P(X \in A) \approx \frac{1}{n}\sum_{i=1}^{n} I_A(X_i)$$

Example: Estimating π

- Consider a unit circle inside a 2×2 square.
- A random point (x, y) lies inside the circle if:

$$x^2 + y^2 \leq 1$$

- The probability of falling inside the circle is: $P = \frac{\pi}{4}$
- Estimate π via simulation:

$$\pi \approx 4\frac{\text{points inside circle}}{\text{total points}}$$

- Monte Carlo works well for complex probability problems.
- Easily scales to high-dimensional settings.

How Large is the Error? — Central Limit Theorem

- Consider i.i.d. variates $\xi_i = h(X_i)$ with

$$\mathbb{E}[\xi_i] = \mu, \qquad \mathbb{V}[\xi_i] = \sigma^2$$

- Define the sample mean

$$\hat{\mu}_n = \frac{1}{n}\sum_{i=1}^{n} h(X_i)$$

- As n increases, the normalized deviation

$$e_n := \frac{\hat{\mu}_n - \mu}{\sigma/\sqrt{n}}$$

- Converges in distribution to the standard normal distribution

$$e_n \Rightarrow \mathcal{N}(0,1)$$

- Thus the sample mean becomes approximately normally distributed regardless of the original distribution.

Error Estimation for Monte Carlo

- By the CLT:

$$\hat{\mu}_n \sim \mathcal{N}\left(\mu, \frac{\sigma^2}{n}\right)$$

- The uncertainty of $\hat{\mu}_n$ is measured by its standard deviation

$$\frac{\sigma}{\sqrt{n}}$$

- Since σ is usually unknown, we estimate it using the sample variance

$$\hat{\sigma}_n = \sqrt{\frac{1}{n}\sum_{i=1}^{n} h_i^2 - \left(\frac{1}{n}\sum_{i=1}^{n} h_i\right)^2}$$

- The Monte Carlo standard error becomes

$$\epsilon_n = \frac{\hat{\sigma}_n}{\sqrt{n}}$$

Monte Carlo Convergence

- Monte Carlo error decreases at rate

$$O\left(\frac{1}{\sqrt{n}}\right)$$

- To reduce the error by a factor of 10, the number of samples must increase by a factor of 100.
- The standard error is the standard deviation of the estimator $\hat{\mu}_n$.
- For a normal distribution $\mathcal{N}(\mu, \sigma^2)$:
  - 68.27% of values lie within 1σ
  - 95% within 2σ
  - 99.7% within 3σ

Monte Carlo Confidence Interval

- Consider the normalized estimator

$$Z_n = \frac{\hat{\mu}_n - \mu}{\epsilon_n}$$

- Approximately follows

$$Z_n \approx \mathcal{N}(0, 1)$$

- A two-sided confidence interval with level $1 - \delta$ satisfies

$$P\left(-N^{-1}(1 - \delta/2) \leq Z_n \leq N^{-1}(1 - \delta/2)\right) = 1 - \delta$$

- Rearranging gives the confidence interval

$$\left[\hat{\mu}_n - N^{-1}(1 - \delta/2)\epsilon_n,\ \hat{\mu}_n + N^{-1}(1 - \delta/2)\epsilon_n\right]$$

Confidence Interval Interpretation

- The interval covers the true value μ with probability $1 - \delta$.
- The interval is random because it depends on $\hat{\mu}_n$ and ϵ_n.
- When ϵ_n is estimated using $\hat{\sigma}_n/\sqrt{n}$, the distribution becomes Student's t.

Rule of Thumb

- For $\delta = 0.05$:

$$N^{-1}(1 - \delta/2) \approx 1.96$$

- Thus a 95% confidence interval is

$$\hat{\mu}_n \pm 1.96\,\epsilon_n$$

- Other common levels:
  - 90%: 1.645
  - 99%: 2.576

Generation of Random Process X_i

- The only non-trivial part of Monte Carlo simulation is generating X_i.
- In the expectation $\mathbb{E}[h(X_i)]$:
  - h is the payoff function.
  - X_i represents the underlying asset state.
- The distribution of X_i may not be known in closed form.
- Under the Black–Scholes model:

$$\frac{dS}{S} = (r - q)dt + \sigma dW_t$$

- Under the local volatility model:

$$\frac{dS}{S} = (r - q)dt + \sigma(S, t)dW_t$$

Random Number Generators

- Random numbers from non-uniform distributions can be generated from uniform samples.
- Common methods:
  - CLT approximation
  - Box–Muller transform
  - Inverse transform sampling
- Computers are deterministic, so they produce pseudo-random numbers.
- True randomness requires external sources.
- Pseudo-random numbers are sufficient for most Monte Carlo applications.

Desired Properties of Random Numbers in QF

- Samples should mimic i.i.d. draws from the target distribution.
- Pseudo-random numbers work well for Monte Carlo integration.
- Reproducibility is important for debugging and validation.
- No PRNG is perfect, so multiple generators may be used for validation.

PRNG: General Principle

1. **State initialization:** internal seed variables define the generator state.
2. **State transition:** a deterministic recurrence updates the state.
3. **Number generation:** a transformation maps the state to a uniform random variable $U(0, 1)$.

- PRNGs generate sequences that appear random while remaining deterministic and reproducible.

PRNG: Middle-Square Method

- One of the earliest pseudo-random number algorithms (von Neumann).
- State variable: x_i
- Iteration rule: square the number and extract the middle digits.
- Example with seed $x_0 = 0.9876$

$$x_0 = 0.9876 \quad x_0^2 = 0.97535376 \quad x_1 = 0.5353 \quad x_1^2 = 0.28654609 \quad x_2 = 0.6546$$

Middle-Square Method: Issue

- A major drawback of the Middle-Square Method is its tendency to enter short periodic cycles or collapse to zero.
- This makes it unreliable for generating long sequences of random numbers.
- Example 1: With seed 0.9876, the sequence starts repeating from x_{46}, producing only four distinct values before cycling:

$$0.96 \to 0.16 \to 0.56 \to 0.36$$

- Example 2: With seed 0.5649, the sequence collapses to zero at x_{84} and remains stuck thereafter.

Congruential Generators

- Congruential generators update the state variable using the recurrence

$$m_{n+1} = (am_n + c) \bmod M$$

- Values are scaled to $[0, 1]$ by

$$x_n = \frac{m_n}{M - 1}$$

- To avoid collapse to a fixed point:
  - a and M must be co-prime
  - $c \neq 0$ prevents absorption at zero
- When $c = 0$, the generator becomes a **linear congruential generator (LCG)**.
- In this case the sequence remains non-trivial provided
  - a and M are co-prime
  - $m_0 \neq 0$
- These conditions ensure a long cycle before repetition.

Congruential Generators: Example

- For small M, the generator quickly enters a periodic cycle.
- Example:

$$M = 11, \quad a = 3, \quad c = 0, \quad m_0 = 3$$

- Generated sequence (period 5):

$$9, 5, 4, 1, 3, 9, 5, 4, 1, \ldots$$

- The maximum possible period is $M - 1$, but this is not guaranteed.
- With sufficiently large M, repetitions become negligible for practical use.
- Choice of parameters is critical.
- Example early IBM generator:

$$a = 65539, \quad M = 2^{31}, \quad m_0 = 1$$

- Later found to have poor statistical properties.
- Improved parameters:

$$a = 5^{17}, \quad M = 2^{40}, \quad m_0 = 1$$

- Achieves period 2^{38}.

Congruential Generators: Extensions

1. **Rand0: Minimal Standard Generator**
   - LCG with

$$a = 16807, \quad M = 2^{31} - 1$$

   - Provides a long period and reasonable statistical properties.
2. **Rand1: Shuffled Enhancement**
   - Improves Rand0 using a shuffle table.
   - Previously generated values are stored and reused to reduce correlation.
   - Example:
     - Generated numbers: $17, 42, 8, 91, 35, 60, 14, \ldots$
     - Shuffle table initialized with: $[17, 42, 8, 91]$
     - New value 35 selects entry 2: output 42, replace by 35
     - New value 60 selects entry 4: output 91, replace by 60
     - New value 14 selects entry 1: output 17, replace by 14

Congruential Generators: Extensions (Rand2)

- Combines two independent LCGs to extend the period.

$$Y_n = (X_n + Z_n) \bmod M$$

- Helps avoid short cycles and improves distribution properties.
- Example sequences:

$$X_n : 12, 45, 31, 8, 27, \ldots$$

$$Z_n : 7, 19, 40, 22, 11, \ldots$$

$$Y_n = (X_n + Z_n) \bmod 50$$

$$Y_1 = 19, \quad Y_2 = 14, \quad Y_3 = 21, \quad Y_4 = 30$$

- Combined sequence:

$$19, 14, 21, 30, 38, \ldots$$

Mersenne Twister (Matsumoto & Nishimura, 1998)

- Designed to eliminate flaws of earlier generators.
- Provides extremely long period:

$$2^{19937} - 1$$

- Uses a twisting transformation to mix bits and improve randomness.

$$x_{k+n} = x_{k+m} \oplus ((x_k^{upper} \parallel x_{k+1}^{lower})A)$$

- $\oplus$ denotes bitwise XOR.
- $\parallel$ denotes concatenation of bits.
- A is a fixed transformation matrix.

Mersenne Twister (MT19937) and Equidistribution

- Equidistribution means generated vectors fill the k-dimensional space evenly.

$$(u_1, u_2), (u_3, u_4), \ldots$$

- In general: $(u_i, u_{i+1}, \ldots, u_{i+k-1})$

- MT19937 achieves 623-dimensional equidistribution.
- Much better than LCGs which typically achieve only a few dimensions.

LCGs are Weak in High Dimensions

- Successive k-dimensional points generated by LCGs lie on parallel hyperplanes.
- Marsaglia's theorem:

$$\text{At most } (k!)^{1/k} \text{ hyperplanes}$$

- Example:
  - $k = 3$: about 2.3 planes
  - $k = 6$: about 2.99 planes
- Thus patterns emerge in higher dimensions.
- This is undesirable for Monte Carlo simulation.

Uniform to Normal via Central Limit Theorem

- Transform uniform samples $U(0,1)$ into normal variables.
- Using CLT:

$$\frac{\hat{\mu}_n - \mu}{\sigma/\sqrt{n}} \sim \mathcal{N}(0,1)$$

- Variance of uniform distribution:

$$\int_0^1 (x-0.5)^2 dx = \frac{1}{12}$$

- Generate 12 uniform samples u_i and compute

$$m_k = \sum_{i=1}^{12} u_i - 6$$

- The sum of uniform variables approximates a normal distribution.
- Subtracting 6 centers the distribution around zero.

Uniform To Normal Distribution: Box–Muller Method

- The CLT-based method for generating normal variables lacks precision, especially in the tails.
- The Box–Muller method is a simple and efficient algorithm to transform uniform random variables into normally distributed variables.
- It uses a property of the bivariate normal distribution.
- If Z_1, Z_2 are independent standard normal variables, then

$$R = \sqrt{Z_1^2 + Z_2^2}$$

- follows an exponential distribution with mean 2:

$$P(R \le r^2) = 1 - e^{-r^2/2}$$

- Conditional on $R = r$, the point (Z_1, Z_2) is uniformly distributed on the circle of radius r.
- The joint PDF of (Z_1, Z_2) is

$$f_{Z_1,Z_2}(z_1, z_2) = \frac{1}{2\pi} e^{-\frac{1}{2}(z_1^2+z_2^2)}$$

Derivations

- Represent the variables in polar coordinates

$$Z_1 = R\cos\theta, \qquad Z_2 = R\sin\theta$$

- The Jacobian of the transformation is R.
- The joint PDF of (R, θ) becomes

$$f_{R,\theta} = \frac{1}{2\pi} e^{-r^2/2} r, \qquad 0 \le r < \infty, \quad 0 \le \theta \le 2\pi$$

- Thus
  - θ is uniform on $[0, 2\pi]$
  - $f_R(r) = re^{-r^2/2}$

Probability Integral Transform

- If $U \sim U(0,1)$ and F_X is the CDF of X, then

$$X = F_X^{-1}(U)$$

- produces the random variable with distribution F_X.

Derivation

- If $U \sim U(0,1)$

$$P(U \le y) = y$$

- Using the exponential CDF

$$1 - e^{-r^2/2} = U$$

- solving gives

$$r = \sqrt{-2\ln U}$$

Box–Muller Algorithm

1. Generate $U_1, U_2 \sim U(0,1)$
2. $R \leftarrow -2\log(U_1)$
3. $V \leftarrow 2\pi U_2$
4. Compute

$$Z_1 = \sqrt{R}\cos V, \qquad Z_2 = \sqrt{R}\sin V$$

Acceptance–Rejection Method

- A general method to sample from a target distribution $f(x)$ using an easier proposal distribution $g(x)$.
- Suppose

$$f(x) \le c\, g(x)$$

- Procedure:
  - Sample x from $g(x)$
  - Accept with probability $\frac{f(x)}{cg(x)}$
  - Otherwise reject and repeat
- Practically: draw $u \sim U(0,1)$ and accept if

$$u \le \frac{f(x)}{cg(x)}$$

Why Acceptance–Rejection Works

- Let Y denote the accepted samples.
- For any set A

$$P(Y \in A) = \frac{P(x \in A, u \le f(x)/cg(x))}{P(u \le f(x)/cg(x))}$$

- The denominator equals

$$P\left(u \le \frac{f(x)}{cg(x)}\right) = \int \frac{f(x)}{cg(x)} g(x)dx = \frac{1}{c}$$

- Therefore

$$P(Y \in A) = \int_A f(x)\, dx$$

- Thus Y has density $f(x)$.
- Acceptance rate is $1/c$.

Acceptance–Rejection Algorithm

1. repeat
2. Generate $x \sim g(x)$
3. Generate $u \sim U(0,1)$
4. until $u \le f(x)/cg(x)$
5. return x

Polar Rejection Method

- A modified version of Box–Muller that avoids computing sin and cos.
- Uses acceptance–rejection to generate points uniformly inside the unit disk.

1. repeat
2. generate $u_1, u_2 \sim U(0,1)$
3. transform

$$u_1 \leftarrow 2u_1 - 1, \qquad u_2 \leftarrow 2u_2 - 1$$

1. compute

$$x = u_1^2 + u_2^2$$

1. until $x \le 1$
2. compute

$$y = \sqrt{-2\ln x}$$

$$Z_1 = y\frac{u_1}{\sqrt{x}}, \qquad Z_2 = y\frac{u_2}{\sqrt{x}}$$

- Faster than Box–Muller despite the acceptance step.
- Acceptance probability is $\pi/4$.

Lecture 8

Inverse Transformation Method

- A fundamental approach for generating random samples from a target distribution transforms uniform random variables using the inverse of the cumulative distribution function (CDF).

Given a random variable X with CDF:

$$P(X \le x) = CDF(x)$$

The inverse transformation method sets

$$X = CDF^{-1}(u), \quad u \sim U(0,1)$$

- To generate a sample x, draw $u \sim U(0,1)$ and compute

$$x = CDF^{-1}(u)$$

- This method is used, for example, to generate exponential variables in the Box–Muller method.

Inverse Transformation Method: Derivation

- Express the probability for X in terms of U:

$$P(X \le x) = P(CDF^{-1}(U) \le x) = P(U \le CDF(x))$$

- For $U \sim U(0,1)$ the CDF is

$$P(U \le u) = u, \quad u \in [0,1]$$

Therefore

$$P(X \le x) = P(U \le CDF(x)) = CDF(x)$$

Inverse Transformation Method — Considerations

- The inverse CDF is not always one-to-one.
- Situations may arise where:
  - many-to-one mappings occur (jumps in CDF)
  - one-to-many mappings occur (flat CDF)
- In such cases predefined conventions are required when applying inverse transforms.

Inverse Transform Method for Normal Distribution

- The normal CDF has no closed-form inverse.

Typical procedure:

- Use an analytical approximation for $N^{-1}(x)$ as an initial estimate.
- Apply a root-finding algorithm to refine the solution.

Efficiency considerations:

- This can be computationally expensive due to iterative methods.
- Alternatives such as the **Box–Muller transform** are commonly used.

Advantages:

- The normal CDF $N(x)$ is strictly increasing, preserving monotonicity.
- Only one uniform random number is required per normal sample.

Pricing European Options with Monte Carlo

Under the Black–Scholes model:

$$\frac{dS_t}{S_t} = r dt + \sigma dW_t$$

Since S_T has a closed-form solution:

$$S_T = S_0 e^{(r-q-\frac{1}{2}\sigma^2)T + \sigma W_T}$$

- Only the terminal value W_T needs to be simulated.
- This avoids simulating the entire Brownian path.

Properties of Brownian motion:

- $W_0 = 0$
- $W_t \sim \mathcal{N}(0, t)$

Thus

$$W_T = \sqrt{T} Z, \quad Z \sim \mathcal{N}(0, 1)$$

Models without Closed-Form Solutions

- Black–Scholes is an exception; most SDEs lack closed-form solutions.

Example: local volatility model

$$dS = S(r-q)dt + S\sigma(S,t)dW_t$$

- Since $\sigma(S,t)$ depends on the state, no analytical solution exists.

General SDE:

$$dX(t) = a(X(t))dt + b(X(t))dW(t)$$

Integrating:

$$X(t_{i+1}) = X(t_i) + \int_{t_i}^{t_{i+1}} a(X(u))du + \int_{t_i}^{t_{i+1}} b(X(u))dW_u$$

- Numerical methods such as Euler–Maruyama are required.

Euler Discretization Scheme

- For time step Δt:

$$\int_t^{t+\Delta t} a(X(u))du = a(X(t))\Delta t + O(\Delta t^2)$$

$$\int_t^{t+\Delta t} b(X(u))dW_u = b(X(t))\Delta W = b(X(t))\sqrt{\Delta t} Z + O(\Delta t)$$

where $Z \sim N(0,1)$.

Step update:

$$X(t+\Delta t) = X(t) + a(X(t))\Delta t + b(X(t))\sqrt{\Delta t} Z$$

- Drift truncation error: $O(\Delta t)$
- Diffusion truncation error: $O(\sqrt{\Delta t})$

Overall convergence order:

$$O(\sqrt{\Delta t})$$

Euler Scheme for Black–Scholes

$$S_{t+\Delta t} = S_t(1 + \mu\Delta t + \sigma\Delta W_t)$$

Example simulation:

- $S_0 = 10$
- $\sigma = 30\%$
- $\mu = 10\%$
- horizon = 5 years

Change of State Variable

- The state variable is the variable tracked during Monte Carlo simulation.
- It does not need to be the asset price itself.

Using log-transformed state variable:

$$X_t = \log S_t$$

Ito expansion gives

$$dX_t = \left(\mu - \frac{1}{2}\sigma^2\right) dt + \sigma dW_t$$

Euler update:

$$X_{t+\Delta t} = X_t + \left(\mu - \frac{1}{2}\sigma^2\right)\Delta t + \sigma\Delta W_t$$

Reconstruct the price:

$$S_{t+\Delta t} = e^{X_{t+\Delta t}} = S_t e^{(\mu - \frac{1}{2}\sigma^2)\Delta t + \sigma\Delta W_t}$$

- This follows from the linear diffusion of X.

Local Volatility Model Diffusion

Using the same state variable X:

$$dX_t = \left(\mu - \frac{1}{2}\sigma(S,t)^2\right) dt + \sigma(S,t)dW_t$$

Euler discretization:

$$X_{t+\Delta t} = X_t + \left(\mu - \frac{1}{2}\sigma(X,t)^2\right)\Delta t + \sigma(X,t)\Delta W_t$$

- Unlike geometric Brownian motion, no closed-form solution exists.
- Discretization introduces error of order

$$O(\sqrt{\Delta t})$$

Milstein Discretization Scheme

- Higher–order convergence in discretization can be achieved using the Milstein scheme.
- This scheme is particularly popular in Monte Carlo simulations for improving accuracy.
- The goal is to achieve $O(\Delta t)$ convergence. Since the drift term is already $O(\Delta t)$, we only need to refine the diffusion term.
- In the Euler scheme, the diffusion term is approximated using $b(X(t))$ over the period $t \to t+\Delta t$, which introduces truncation error.
- The Milstein scheme corrects this by incorporating an additional term to capture the stochastic integral more accurately.

Milstein Discretization Derivation

Consider the process

$$dX(t) = a(X(t))dt + b(X(t))dW(t)$$

The Euler discretization reads

$$X(t+\Delta t) = X(t) + a(X(t))\Delta t + b(X(t))\Delta W_t$$

- The issue is that this assumes $b(X_t)$ is constant over $[t, t+\Delta t]$.
- For higher accuracy we account for the fact that $b(X_t)$ is also stochastic.

Using Itô's lemma and $(dX)^2 = b^2 dt$:

$$db(X(t)) = \frac{\partial b}{\partial X} dX + \frac{1}{2}\frac{\partial^2 b}{\partial X^2} dX^2$$

$$= b'(a\,dt + b\,dW_t) + \frac{1}{2} b'' b^2 dt$$

$$= (ab' + \tfrac{1}{2} b^2 b'')dt + b(X(t))b'(X(t))dW_t$$

Approximating the Diffusion Term

$$b(X(u)) = b(X(t)) + \int_t^u b(X(s))b'(X(s))dW_s + \int_t^u c(X(s))ds$$

- The integral $\int_t^u c(X(s))ds$ is of order $O(\Delta t)$ and can be omitted.

Thus

$$b(X(u)) \approx b(X(t)) + \int_t^u b(X(s))b'(X(s))dW_s$$

Approximating the stochastic integral:

$$\int_t^u b(X(u))b'(X(u))dW_u = b(X(t))b'(X(t))(W_u - W_t) + O(u-t)$$

Hence

$$b(X(u)) = b(X(t)) + b(X(t))b'(X(t))(W_u - W_t) + O(u-t)$$

Milstein Correction

Recall

$$X(t_{i+1}) = X(t_i) + \int_{t_i}^{t_{i+1}} a(X(u))du + \int_{t_i}^{t_{i+1}} b(X(u))dW_u$$

Then

$$\int_{t_i}^{t_{i+1}} b(X(u))dW_u = b(X(t))(W_{t+\Delta t} - W_t) + \frac{1}{2} b(X(t))b'(X(t))((W_{t+\Delta t} - W_t)^2 - \Delta t)$$

Equivalently,

$$\int_{t_i}^{t_{i+1}} b(X(u))dW_u = b(X(t))\sqrt{\Delta t}Z + \frac{1}{2}b(X(t))b'(X(t))\Delta t(Z^2-1)$$

where $Z \sim N(0,1)$.

Itô Isometry

Using Itô's formula for W_u^2:

$$d(W_u^2) = 2W_u dW_u + du$$

Thus

$$W_u dW_u = \frac{1}{2}d(W_u^2) - \frac{1}{2}du$$

Integrating from t to $t+\Delta t$:

$$\int_t^{t+\Delta t} W_u dW_u = \frac{1}{2}(W_{t+\Delta t}^2 - W_t^2) - \frac{1}{2}\Delta t$$

Milstein Scheme for Local Volatility

In a local volatility model

$$b(X,t) = \sigma(X,t)$$

and

$$b'(X) = \frac{\partial \sigma(X,t)}{\partial X}$$

Euler scheme:

$$X_{t+\Delta t} \approx X_t + \left(\mu - \frac{\sigma(X,t)^2}{2}\right)\Delta t + \sigma(X,t)\Delta W_t$$

Milstein correction:

$$\frac{1}{2}\sigma(X,t)\frac{\partial \sigma(X,t)}{\partial X}(\Delta W_t^2 - \Delta t)$$

Final scheme:

$$X_{t+\Delta t} = X_t + \left(\mu - \frac{\sigma(X,t)^2}{2}\right)\Delta t + \sigma(X,t)\Delta W_t + \frac{\sigma(X,t)}{2}\frac{\partial \sigma(X,t)}{\partial X}(\Delta W_t^2 - \Delta t)$$

Example: Milstein with State Variable S

Consider the local volatility SDE

$$dS_t = \mu S_t dt + \sigma(S_t,t)S_t dW_t$$

Milstein scheme:

$$S_{t+\Delta t} = S_t + \mu S_t \Delta t + \sigma(S_t,t)S_t\Delta W_t + \frac{1}{2}b(S_t)b'(S_t)(\Delta W_t^2 - \Delta t)$$

Here

$$b(S_t) = \sigma(S_t,t)S_t$$

$$b'(S_t) = \sigma(S_t,t) + S_t\frac{\partial \sigma(S_t,t)}{\partial S}$$

Thus

$$\frac{1}{2}\sigma(S_t,t)S_t\left(\sigma(S_t,t) + S_t\frac{\partial \sigma(S_t,t)}{\partial S}\right)(\Delta W_t^2 - \Delta t)$$

If σ is constant (Black–Scholes):

$$S_{t+\Delta t} = S_t\left(1 + \mu\Delta t + \sigma\Delta W_t + \frac{\sigma^2}{2}(\Delta W_t^2 - \Delta t)\right)$$

Monte Carlo Method — Pros and Cons

Pros:

- Easy to implement.
- Highly extensible to complex models and path dependencies.
- Performs well in high-dimensional problems.

Cons:

- Monte Carlo noise requires many simulations for accuracy.
- Slower than PDE or tree methods in low dimensions.
- Limited efficiency for American options (requires LSMC).

Two-Factor Monte Carlo: Correlated Brownian Motions

When simulating multiple factors, correlated Brownian motions must be constructed.

Correlation of increments:

$$\rho = \frac{\mathrm{Cov}(\Delta W_1, \Delta W_2)}{\sqrt{\mathrm{Var}(\Delta W_1)\mathrm{Var}(\Delta W_2)}} = \frac{E(\Delta W_1 \Delta W_2)}{dt}$$

Since

$$E(\Delta W_i) = 0, \quad Var(\Delta W_i) = dt$$

we obtain

$$\rho = \frac{E(\Delta W_1 \Delta W_2)}{dt}$$

Let

$$\Delta W_1 = \xi_1\sqrt{dt}, \qquad \Delta W_2 = \xi_2\sqrt{dt}$$

with

$$E(\xi_1\xi_2) = \rho$$

Correlation of the Increments of Two Brownian Motions

- Represent the correlation matrix as

$$\mathbf{C} = \begin{pmatrix} 1 & \rho \\ \rho & 1 \end{pmatrix}$$

Random samples for ξ_1 and ξ_2 (to generate correlated Brownian increments):

$$\hat{\xi}_1 = [\hat{\xi}_{1,0}, \hat{\xi}_{1,1}, \ldots, \hat{\xi}_{1,n}]$$

$$\hat{\xi}_2 = [\hat{\xi}_{2,0}, \hat{\xi}_{2,1}, \ldots, \hat{\xi}_{2,n}]$$

In terms of expectations:

$$\begin{pmatrix} \hat{\xi}_1 \\ \hat{\xi}_2 \end{pmatrix}\begin{pmatrix} \hat{\xi}_1^\top & \hat{\xi}_2^\top \end{pmatrix} = \begin{pmatrix} \hat{\xi}_1\cdot\hat{\xi}_1 & \hat{\xi}_1\cdot\hat{\xi}_2 \\ \hat{\xi}_2\cdot\hat{\xi}_1 & \hat{\xi}_2\cdot\hat{\xi}_2 \end{pmatrix} = \begin{pmatrix} n & n\rho \\ n\rho & n \end{pmatrix} = n\mathbf{C}$$

Correlating Standard Normal Vectors

- For $-1 < \rho < 1$, the matrix $\mathbf{C}$ is invertible.
- Decompose it using Cholesky factorization:

$$\mathbf{C} = \mathbf{L}\mathbf{L}^\top$$

We have

$$\begin{pmatrix} \hat{\xi}_1 \\ \hat{\xi}_2 \end{pmatrix}\begin{pmatrix} \hat{\xi}_1^\top & \hat{\xi}_2^\top \end{pmatrix} = n\mathbf{L}\mathbf{L}^\top$$

Thus

$$\mathbf{L}^{-1}\begin{pmatrix} \hat{\xi}_1 \\ \hat{\xi}_2 \end{pmatrix}\begin{pmatrix} \hat{\xi}_1^\top & \hat{\xi}_2^\top \end{pmatrix}(\mathbf{L}^\top)^{-1} = n\mathbf{I}_2$$

where $\mathbf{I}_2$ is the 2×2 identity matrix.

Generating Correlated Normals

Let

$$\begin{pmatrix} \zeta_1 \\ \zeta_2 \end{pmatrix} = \mathbf{L}^{-1}\begin{pmatrix} \hat{\xi}_1 \\ \hat{\xi}_2 \end{pmatrix}$$

Then ζ_1, ζ_2 are uncorrelated since

$$\begin{pmatrix} \zeta_1 \\ \zeta_2 \end{pmatrix}\begin{pmatrix} \zeta_1^\top & \zeta_2^\top \end{pmatrix} = n\mathbf{I}$$

Therefore, if we generate independent standard normals ζ_1, ζ_2, we obtain correlated normals by

$$\begin{pmatrix} \hat{\xi}_1 \\ \hat{\xi}_2 \end{pmatrix} = \mathbf{L}\begin{pmatrix} \zeta_1 \\ \zeta_2 \end{pmatrix}$$

Cholesky Decomposition

- If a matrix is symmetric and positive definite, it admits a Cholesky decomposition:

$$\mathbf{C} = \mathbf{L}\mathbf{L}^\top$$

where $\mathbf{L}$ is lower triangular.

- For correlation matrices this holds when $-1 < \rho < 1$.

To verify positive definiteness:

$$\mathbf{x}^\top\mathbf{C}\mathbf{x} = \frac{1}{n}\mathbf{x}^\top\begin{pmatrix} \hat{\xi}_1 \\ \hat{\xi}_2 \end{pmatrix}\begin{pmatrix} \hat{\xi}_1^\top & \hat{\xi}_2^\top \end{pmatrix}\mathbf{x} = \frac{\|y\|}{n} \geq 0$$

where

$$y = [\hat{\xi}_1, \hat{\xi}_2]\mathbf{x}$$

Thus $\mathbf{C}$ is semi-positive definite and strictly positive definite when $\rho \neq \pm 1$.

Generating Two Correlated Brownian Motions

Procedure:

1. Generate independent standard normals ζ_1, ζ_2.
2. Compute Cholesky decomposition $\mathbf{C} = \mathbf{L}\mathbf{L}^\top$.
3. Generate correlated normals

$$\begin{pmatrix} \hat{\xi}_1 \\ \hat{\xi}_2 \end{pmatrix} = \mathbf{L}\begin{pmatrix} \zeta_1 \\ \zeta_2 \end{pmatrix}$$

- The same procedure extends to m correlated random variables.

Cholesky Decomposition Algorithm
Example for a 3×3 symmetric matrix

$$\mathbf{C} = \begin{pmatrix} C_{11} & C_{12} & C_{13} \\ C_{12} & C_{22} & C_{23} \\ C_{13} & C_{23} & C_{33} \end{pmatrix}$$

Factorization

$$\mathbf{C} = \mathbf{L}\mathbf{L}^\top$$

$$\mathbf{L} = \begin{pmatrix} L_{11} & 0 & 0 \\ L_{21} & L_{22} & 0 \\ L_{31} & L_{32} & L_{33} \end{pmatrix}$$

Algorithm:

1. Compute L_{11} from the top-left element.
2. Move down the first column to compute L_{21}, L_{31}.
3. Proceed diagonally to compute L_{22}.
4. Continue column by column.

Python implementation:

$$L = \texttt{np.linalg.cholesky(C)}$$

Example of Correlated Brownian Motions
- Using the above procedure we can generate Brownian motions with different correlation coefficients ρ.

Lecture 9

Generic Monte Carlo Framework
- Similar to the Binomial Tree framework, the Monte Carlo (MC) framework can be structured in a modular way to price financial derivatives.
- Monte Carlo methods provide greater flexibility than tree-based methods, particularly for path-dependent products and exotic options.

Core Components
- **Market Model:** Defines interest rates, volatility, and asset dynamics.
- **Random Number Generator (RNG):** Produces stochastic inputs required for simulating asset paths.
- **Monte Carlo Product:** Represents the derivative contract including payoff and contract terms.
- **Diffusion Model:** Describes the stochastic evolution of asset prices (e.g., Black–Scholes, Heston).
- **MC Pricer:** Central engine that simulates paths and computes expected payoffs.

Monte Carlo Framework: Main Steps
- **Handling Multi-Asset Trades**
  - Market model must account for correlation between underlying assets.
- **Generating Correlated Brownian Motions**
  - RNG generates independent Brownian motions.
  - They are transformed using the correlation matrix to obtain correlated factors.
- **Diffusion Model**
  - Maps each Brownian motion to an asset price process.
  - Each asset evolves according to a chosen stochastic differential equation (SDE).
- **Simulating Market Realizations**
  - Each Monte Carlo path represents one possible future market scenario.
- **Trade Valuation**
  - Discounted payoff computed for each simulated path.
  - Corresponds to evaluating
    $$h(X_i)$$
    in the expected value
    $$\mathbb{E}[h(X)]$$
  - Final estimate obtained by averaging across simulation paths.

Diffusion Model Requirements
A diffusion model must provide:
- `num_factors()` Returns the number of Brownian motions required.
- `get_time_steps(event_dates)` Returns the time grid used for diffusion.
- `diffuse(dts, bs)` Returns a function that interpolates the simulated asset price path.

Trade Payoff Functions
Once price paths are simulated:
- The trade computes discounted payoff h = trade.discounted_mc_payoff(fobs)
- This evaluates the present value for the simulated market scenario.
- Trade must also provide:
  - Assets involved in the payoff:
    $$trade.asset_names()$$
  - Relevant event dates:
    $$trade.all_dates()$$

Example: Spread Option
- Payoff depends on the difference between two underlying assets.

European spread option with maturity T and strike K:
- Call payoff
  $$\max(S_1(T) - S_2(T) - K, 0)$$
- Put payoff
  $$\max(K - (S_1(T) - S_2(T)), 0)$$
- Requires simulation of two correlated stochastic processes.
- No general closed-form solution in Black–Scholes.

Variance Reduction Techniques
Monte Carlo variance can be reduced using:
- Antithetic sampling
- Variates recycling
- Control variates
- Stratified sampling
- Importance sampling

Antithetic Sampling
Uses negatively correlated sample paths.
For a Brownian motion path W_i, the mirror path $-W_i$ has the same distribution.
Instead of evaluating only

$$h(W_i)$$

we evaluate

$$h(W_i), \quad h(-W_i)$$

Antithetic estimator:

$$\hat{h}_i = \frac{1}{2}(h(W_i) + h(-W_i))$$

Monte Carlo estimate:

$$\frac{1}{n}\sum_{i=1}^{n} \hat{h}_i$$

Variance:

$$Var\left[\frac{1}{2}(h(W_i) + h(-W_i))\right] = \frac{1}{2}Var[h(W_i)] + \frac{1}{2}Cov[h(W_i), h(-W_i)]$$

- Variance reduced when covariance term is negative.
- Often occurs when payoff is monotonic in W.

When Antithetic Sampling Works Best
- Effective for asymmetric payoffs.
- Less effective for symmetric payoffs such as straddles.

Variates Recycling for Greek Estimation
Greeks (risk sensitivities) are often computed via finite differences.
Example: Delta

$$\delta = \frac{\partial PV}{\partial S} \approx \frac{PV(S + \Delta S) - PV(S - \Delta S)}{2\Delta S}$$

Variance of estimator:

$$\frac{1}{4\Delta S^2}\left(Var(PV_{S+\Delta S}) + Var(PV_{S-\Delta S}) - 2Cov(PV_{S+\Delta S}, PV_{S-\Delta S})\right)$$

Variance reduction idea:
- Reuse the same random numbers for both simulations.
- Increases correlation between estimators.
- Reduces overall variance.
- This motivates using reproducible random numbers.

Control Variates
- Goal: estimate the expectation of a function $h(X)$.
- If the expectation of another function $g(X)$ is known analytically, we construct the estimator

$$\mathbb{E}[h(X)] \approx \frac{1}{n}\sum_{i=1}^{n}\left(h(X_i) + \beta(g^* - g(X_i))\right)$$

- $g^* = \mathbb{E}[g(X)]$ is known analytically
- β is a parameter

Variance of the Estimator

$$\mathrm{Var}[h] + \beta^2\mathrm{Var}[g] - 2\beta\,\mathrm{Cov}[h, g]$$

Minimizing w.r.t. β gives

$$\beta = \frac{\mathrm{Cov}[h, g]}{\mathrm{Var}[g]}$$

- The optimal β can be estimated from simulation samples.

Variance Reduction

$$\mathrm{Var}[h] - \frac{\mathrm{Cov}[h, g]^2}{\mathrm{Var}[g]} = \mathrm{Var}[h](1 - \rho^2)$$

- ρ is the correlation between h and g
- Higher correlation $\Rightarrow$ stronger variance reduction

Examples
- Forward contract as control variate for a call option
- Geometric Asian option as control variate for arithmetic Asian option

Arithmetic vs geometric average:

$$A_{\text{arith}} = \frac{1}{n}\sum_{i=1}^{n} S(t_i), \quad A_{\text{geom}} = \left(\prod_{i=1}^{n} S(t_i)\right)^{1/n}$$

- Geometric Asian option has a Black–Scholes closed form.

Stratified Sampling

- Partition the domain into k disjoint strata A_j

$$\mathbb{E}[h(X)] = \sum_{j=1}^{k} \mathbb{E}[h(X \in A_j)]\, P(X \in A_j)$$

- Allocate more samples to regions with higher variance.
- Inverse transform sampling can generate samples for each stratum.
- Choosing strata is problem dependent and difficult in high dimensions.

Importance Sampling

- Change probability measure to concentrate samples in important regions.

$$\mathbb{E}_Q[h(X)] = \int h(X) q(X) dX$$

Rewrite as

$$= \int h(X) \frac{q(X)}{p(X)} p(X) dX = \mathbb{E}_P \left[h(X) \frac{q(X)}{p(X)} \right]$$

- Choose $q(X)$ so important regions are sampled more frequently.
- Requires careful design of sampling distribution.

Example of Importance Sampling

- Estimate

$$I = \int_0^\infty e^{-x} h(x)\, dx$$

where

$$h(x) = \frac{x^2}{1 + x^2}$$

- This equals $\mathbb{E}[h(x)]$ under an exponential distribution with rate 1.
- Large x values are rarely sampled although $h(x) \approx 1$ there.

Use alternative distribution

$$q(x) = e^{-x/2}$$

Estimator

$$\mathbb{E}_{y \sim q} \left[h(y) \frac{e^{-y}}{e^{-y/2}/2} \right]$$

Choosing a Better Distribution

- $q(x) = e^{-x/2}$ has a heavy tail causing unstable importance weights.
- Better candidate should have good tail coverage and not overly heavy tails.

Example:

$$q(x) = \text{Gamma}(\alpha = 3, \beta = 2) = 4x^2 e^{-2x}$$

Summary of Variance Reduction

- Antithetic sampling
- Variates recycling
- Control variates
- Stratified sampling
- Importance sampling
- These reduce variance but do not change MC convergence rate

$$O(n^{-1/2})$$

Monte Carlo vs Quadrature

Quadrature methods can converge faster for smooth functions. Example: trapezoidal rule

$$\alpha \approx \frac{f(0) + f(1)}{2n} + \frac{1}{n} \sum_{i=1}^{n-1} f(i/n)$$

Trapezoidal Scheme

$$x_i = \frac{i}{n}, \quad h = \frac{1}{n}$$

$$T = \sum_{i=1}^{n} T_i = h \sum_{i=1}^{n} \frac{f(x_{i-1}) + f(x_i)}{2}$$

Using Taylor expansion

$$f(x_i) = f(x_{i-1}) + h f'(x_{i-1}) + \frac{h^2}{2} f''(\xi_i)$$

Error on interval

$$E_i = I_i - T_i = \frac{h^3}{6} f''(\eta_i) - \frac{h^3}{4} f''(\xi_i)$$

Total error

$$E = \sum_{i=1}^{n} E_i = O(nh^3) = O(h^2)$$

- Deterministic quadrature can achieve faster convergence than Monte Carlo.

Monte Carlo vs. Quadrature in Higher Dimensions

- The quadrature error rate

$$O(n^{-2/d})$$

  implies that convergence deteriorates as the dimension d increases.
- Monte Carlo maintains a convergence rate

$$O(n^{-1/2})$$

  due to the Central Limit Theorem.
- Hence Monte Carlo becomes attractive for evaluating high-dimensional integrals.

Monte Carlo estimator:

$$\int_A f(\mathbf{X}) d\mathbf{X} \approx \frac{1}{n} \sum_{i=1}^{n} f(\mathbf{X}_i)$$

Quadrature estimator with equal spacing:

$$\int_A f(\mathbf{X}) d\mathbf{X} = \frac{1}{n} \sum_{i=1}^{n} f(\mathbf{X}_i)$$

- Both estimators average function evaluations.
- The key difference lies in how the sample points $\mathbf{X}_i$ are chosen.

Quadrature in Two Dimensions

For a two-dimensional integral:

$$\int_0^1 \int_0^1 f(x, y)\, dx dy$$

Discretizing each dimension into m points gives:

$$n = m^2$$

Quadrature approximation:

$$\int_0^1 \int_0^1 f(x, y)\, dx dy \approx \frac{1}{m^2} \sum_i \sum_j f(x_i, y_j) = \frac{1}{n} \sum_i \sum_j f(x_i, y_j)$$

- This corresponds to evaluating the function on a regular grid.
- Each dimension contributes error $O(m^{-2})$.
- Overall error in 2D becomes

$$O(n^{-1})$$

Quadrature in d Dimensions

If each dimension is discretized with m points:

$$n = m^d$$

The error becomes $O(n^{-2/d})$. Convergence worsens as d increases.

Sampling Strategy

- Quadrature uses structured grid points.
- Monte Carlo uses randomly distributed points.

Why Does This Matter?

- In multidimensional integration, points are effectively projected onto each dimension.
- For quadrature grids:
  - Projections contain repeated values.
  - This reduces sampling diversity.
- For Monte Carlo:
  - Random samples produce distinct projections.
  - Each dimension retains n unique sample values.

Curse of Dimensionality

- Quadrature methods are efficient in low dimensions.
- However the number of required grid points grows exponentially with d.
- This phenomenon is known as the **curse of dimensionality**.

Quasi Monte Carlo (QMC)

- Monte Carlo performs well in high dimensions but convergence remains slow.
- Quasi Monte Carlo improves sampling by using **low-discrepancy sequences**.

Low-discrepancy sequences aim to distribute points uniformly without clustering.

Discrepancy definition:

$$D_N^{(d)} = \sup_{y \in [0,1]^d} \left| \frac{n_{S(y)}}{N} - \prod_{k=1}^{d} y_k \right|$$

where

$$S(y) = [0, y_1] \times [0, y_2] \times \cdots \times [0, y_d]$$

- $n_{S(y)}$ counts sample points in the region $S(y)$.

Interpreting Discrepancy

- Empirical proportion:

$$\frac{n_{S(y)}}{N}$$

- Expected proportion under uniform distribution:

$$\prod_{k=1}^{d} y_k$$

- Low discrepancy implies these two quantities remain close.

QMC Convergence Rate

$$O\left(\frac{(\log n)^d}{n}\right)$$

- For moderate dimension d, this behaves approximately like

$$O(n^{-1})$$

  which is significantly faster than standard Monte Carlo.

Halton Sequence

- Each dimension is assigned a distinct prime base p_i.

Represent index n in base p_i:

$$\gamma(n) = \sum_{k=1}^{m_{n_i}} a_k p_i^{k-1}$$

Digits a_k are the base-p_i representation of n.
The quasi-random number is obtained by reversing digits:

$$u(n) = \sum_{k=1}^{m_{n_i}} a_k p_i^{-k}$$

Example for base $p_1 = 2$:

- $1 \to 1_2 \to 0.1_2 = 0.5$
- $2 \to 10_2 \to 0.01_2 = 0.25$
- $3 \to 11_2 \to 0.11_2 = 0.75$
- $4 \to 100_2 \to 0.001_2 = 0.125$

MT19937 vs Halton

- MT19937: pseudo-random number generator (PRNG).
- Halton: quasi-random sequence (QMC).
- MT19937:
  - General-purpose randomness.
  - Uniform in the long run but with clustering.
- Halton:
  - Low-discrepancy sampling.
  - Points are more evenly distributed.

Quasi Random Numbers in Monte Carlo

- Careful generation of quasi-random numbers is required when used in simulations.
- Each Brownian increment corresponds to one dimension.
- For T time steps, we must sample T-dimensional vectors.

Limitations of Halton

- As dimension increases, base numbers grow larger.
- Discrepancy increases and effectiveness deteriorates.

QMC and Effective Dimension

- Low-discrepancy sequences have limited effective dimensions.
- High-quality dimensions should be assigned to the most influential variables.

Two common constructions:

- **Brownian Bridge Construction**
  - Assign first dimension to terminal time.
  - Subsequent dimensions fill intermediate points.
- **PCA Construction**
  - Construct Brownian motion using principal components.
  - Concentrates variance in early dimensions.